

Multivariate and Advanced Data Analysis

Statistics and Data Analysis

Unit-5

Dr. Pavinder Yadav

Assistant Professor
School of Computer Science
University of Petroleum and Energy Studies
Dehradun



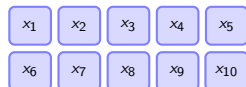
- 1 **Feature Extraction**
 - Principal Component Analysis (PCA)
 - Linear Discriminant Analysis (LDA)
 - SVD, t-SNE, UMAP, and Autoencoders
- 2 **Feature Construction**
 - Mathematical and Polynomial Features
 - Temporal and Text Features
 - Aggregation, Binning, and Encoding
 - Interaction and Domain-Specific Features
- 3 **Feature Selection**
 - Filter Methods
 - Wrapper Methods
 - Embedded Methods
- 4 **Principal Component Analysis (PCA): Introduction and The Mathematics**
- 5 **Factor Analysis**
- 6 **Discriminant Analysis**
- 7 **Kernel PCA**
- 8 **t-SNE (t-distributed Stochastic Neighbor Embedding)**
- 9 **Advanced Feature Engineering for Multivariate Data**

- 1 **Feature Extraction**
 - Principal Component Analysis (PCA)
 - Linear Discriminant Analysis (LDA)
 - SVD, t-SNE, UMAP, and Autoencoders
- 2 **Feature Construction**
 - Mathematical and Polynomial Features
 - Temporal and Text Features
 - Aggregation, Binning, and Encoding
 - Interaction and Domain-Specific Features
- 3 **Feature Selection**
 - Filter Methods
 - Wrapper Methods
 - Embedded Methods
- 4 **Principal Component Analysis (PCA): Introduction and The Mathematics**
- 5 **Factor Analysis**
- 6 **Discriminant Analysis**
- 7 **Kernel PCA**
- 8 **t-SNE (t-distributed Stochastic Neighbor Embedding)**
- 9 **Advanced Feature Engineering for Multivariate Data**

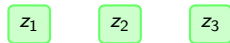
Feature extraction is the process of **transforming original features into a new, smaller set of features** that captures the most important information from the data.

Original (10 features)

- Creates **entirely new features** by combining or transforming the originals.
- The new features are **not** from the original set — they are mathematically derived.
- Reduces dimensionality while preserving key patterns.



Extracted (3 features)



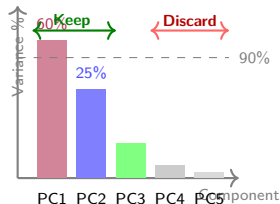
$$z_1 = 0.7x_1 + 0.3x_2 - 0.1x_5$$

Key Distinction

Feature **selection** keeps original features. Feature **extraction** creates **new transformed** features (e.g. PCA components).

Problems It Solves

- **Dimensionality Reduction** — compress hundreds of features to tens.
- **Redundancy Removal** — correlated features combined into independent components.
- **Noise Reduction** — small noisy variations discarded with low-variance components.
- **Visualization** — project high-D data to 2D/3D for human inspection.



Key Limitation

Extracted features lose interpretability — z_1 has no direct physical meaning unlike “income” or “age.”

FEATURE EXTRACTION METHODS

Linear Methods

- PCA
- LDA
- SVD
- Factor Analysis

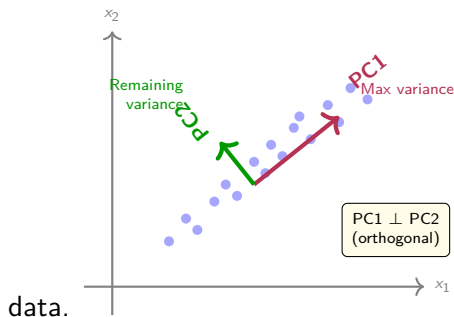
Finds linear combinations
of original features

Non-Linear Methods

- Kernel PCA
- t-SNE
- UMAP
- Autoencoders

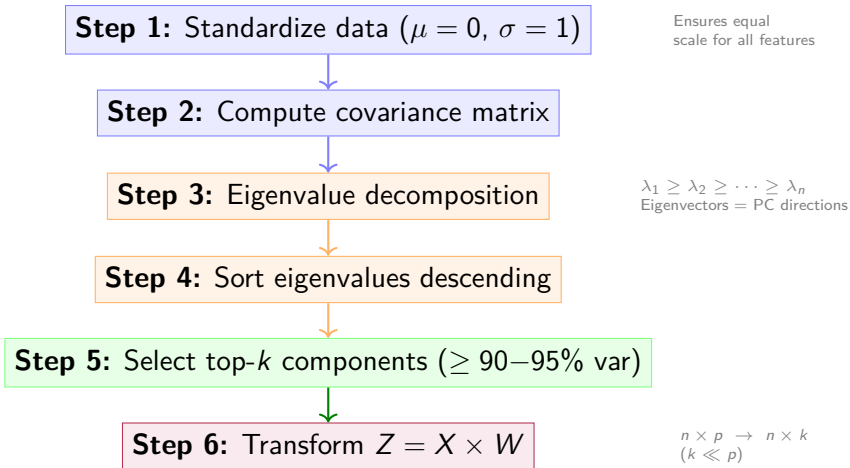
Captures complex, curved
relationships in data

PCA finds new axes called **principal components** that capture the **maximum variance** in the



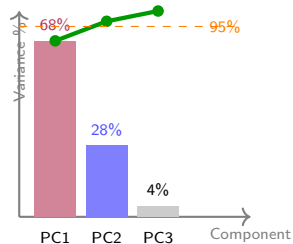
Key Properties

Principal components are **orthogonal** (uncorrelated), ordered by decreasing variance, and are linear combinations of original features.



PC	Var. Ratio	Cumulative
PC1	68.4%	68.4%
PC2	27.6%	96.1%
PC3	3.9%	100%

Choose k where cumulative variance $\geq 90-95\%$.
 Here $k = 2$ captures **96.1%** — excellent compression from 3D to 2D!



Scree Plot Rule

Look for the “elbow” where additional components add negligible variance. Keep components before the elbow.

PCA with scikit-learn

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# Step 1: Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 2: Apply PCA (retain 95% variance)
pca = PCA(n_components=0.95)
X_pca = pca.fit_transform(X_scaled)

print(f"Original: {X.shape[1]} features")
print(f"Reduced: {X_pca.shape[1]} features")
print(f"Variance ratios: {pca.explained_variance_ratio_}")
print(f"Total retained: "
      f"{sum(pca.explained_variance_ratio_):.2%}")
```

Usage Note

`n_components=0.95` automatically selects the number of components to retain 95% of variance.

Use PCA When

- Many correlated features
- Need dimensionality reduction
- Want to remove noise
- Need 2D/3D visualization
- Preprocessing for another model

Avoid PCA When

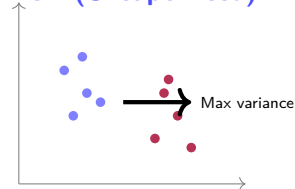
- Interpretability is critical
- Non-linear relationships dominate
- Data has very few features
- Features are already independent
- Data is purely categorical

Remember

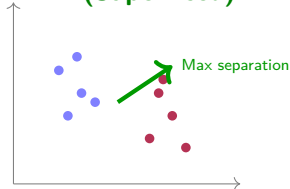
PCA is **unsupervised** — it does not use class labels. It maximizes **total variance**, not class separation.

LDA finds directions that **maximize class separation** rather than total variance.

PCA (Unsupervised)



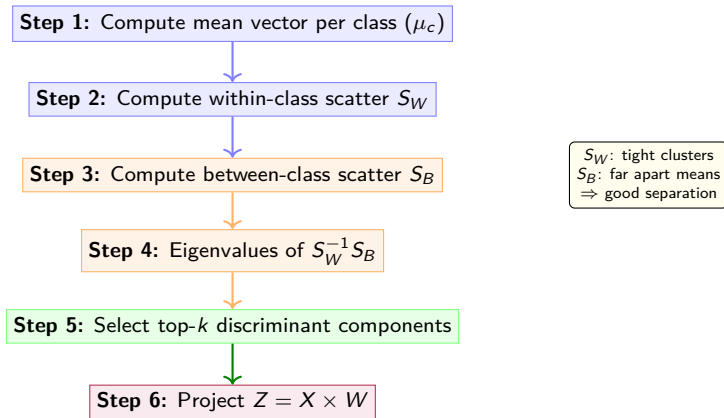
LDA (Supervised)



$$J(W) = \frac{W^T S_B W}{W^T S_W W} \longrightarrow \text{Maximize } \frac{\text{Between-class scatter}}{\text{Within-class scatter}}$$

Constraint

Maximum number of LDA components = $\min(n_{\text{features}}, n_{\text{classes}} - 1)$. For binary classification: at most **1** component.



Examples

For a 5-class problem with 100 features: LDA reduces to at most 4 discriminant components — massive reduction!

LDA with scikit-learn (requires class labels)

```
from sklearn.discriminant_analysis import (  
    LinearDiscriminantAnalysis)  
  
# LDA is SUPERVISED - needs y labels  
lda = LinearDiscriminantAnalysis(n_components=2)  
X_lda = lda.fit_transform(X_scaled, y) # Note: y!  
  
print(f"Original: {X.shape[1]} features")  
print(f"LDA dims: {X_lda.shape[1]}")  
print(f"Explained: {lda.explained_variance_ratio_}")
```

PCA vs LDA

PCA: Unsupervised, maximizes variance.

LDA: Supervised, maximizes class separation.

Tip

For classification tasks, try LDA first — it often produces better low-dimensional representations than PCA.

SVD decomposes any matrix into three matrices. It is the **mathematical foundation behind PCA**.

$$\mathbf{X}_{m \times n} = \mathbf{U}_{m \times m} \mathbf{\Sigma}_{m \times n} \mathbf{V}_{n \times n}^T$$

$\mathbf{X}$
 $m \times n$

=

$\mathbf{U}$
 $m \times m$

$\mathbf{\Sigma}$
 $m \times n$

$\mathbf{V}^T$
 $n \times n$

Row
relationships

Singular values
(importance)

Feature
relationships

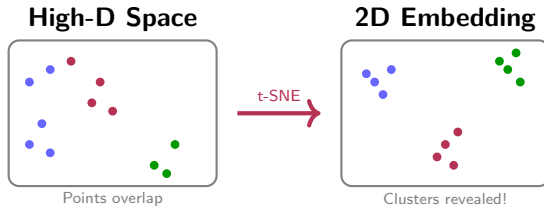
SVD vs PCA

PCA centers data first; SVD works on raw matrix.
TruncatedSVD handles **sparse** matrices efficiently.

Application: LSA

In text mining, SVD on a TF-IDF matrix discovers latent “topics” (Latent Semantic Analysis).

t-SNE maps high-dimensional data to 2D/3D by preserving **local neighborhood structure**.



Key Parameter: Perplexity

Controls neighborhood size. Low (5–10): very local; High (30–50): broader context. Typical: 5–50.

Warning

Use t-SNE for **visualization only**. Do NOT use for model training or interpreting distances between clusters.

UMAP preserves both **local and global structure**, is faster, and scales better.

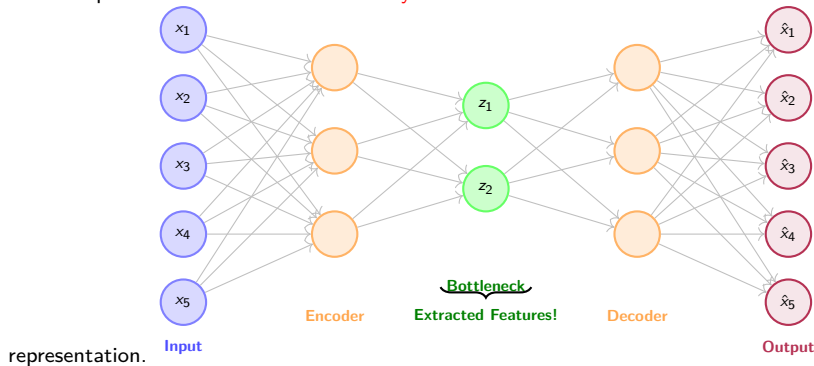
Aspect	t-SNE	UMAP
Speed	Slow $O(n^2)$	Much faster
Scalability	Thousands of points	Millions of points
Global structure	Poor preservation	Better preservation
Reproducibility	Varies across runs	More stable
Use beyond viz	Not recommended	Can use for ML tasks

Key Parameters

`n_neighbors` (local vs. global balance, similar to perplexity) and `min_dist` (how tightly points are packed).

Autoencoders — Neural Network-Based Extraction

An autoencoder compresses data into a **bottleneck layer** and reconstructs it. The bottleneck IS the extracted



Why Autoencoders?

PCA captures only linear patterns. Autoencoders learn **non-linear** compressions and can work with images, sequences, and complex data.

Non-linear extraction methods

```
from sklearn.decomposition import TruncatedSVD
from sklearn.manifold import TSNE
import umap # pip install umap-learn

# TruncatedSVD (works on sparse matrices)
svd = TruncatedSVD(n_components=50)
X_svd = svd.fit_transform(X)

# t-SNE (visualization to 2D)
tsne = TSNE(n_components=2, perplexity=30,
random_state=42, n_iter=1000)
X_tsne = tsne.fit_transform(X_scaled)

# UMAP (faster, preserves global structure)
reducer = umap.UMAP(n_components=2,
n_neighbors=15, min_dist=0.1)
X_umap = reducer.fit_transform(X_scaled)
```

Remember

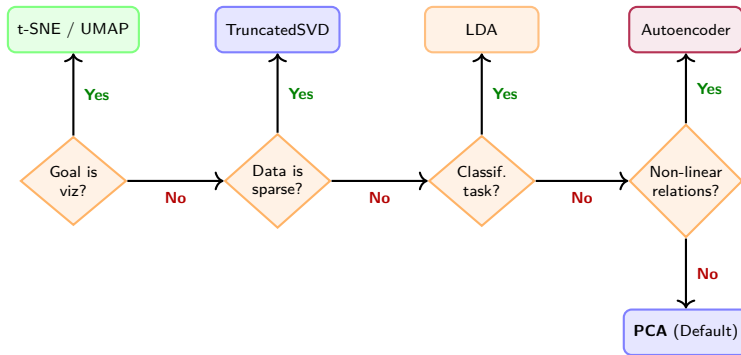
Use TruncatedSVD for sparse data (text, one-hot). Use t-SNE/UMAP primarily for visualization.

Method	Type	Super-vised?	Preserves	Best For	Scalability
PCA	Linear	No	Global variance	General reduction	Excellent
LDA	Linear	Yes	Class separation	Classification	Good
SVD	Linear	No	Matrix structure	Text, sparse data	Excellent
t-SNE	Non-linear	No	Local structure	Visualization only	Poor
UMAP	Non-linear	Both	Local + global	Visualization + ML	Good
Autoencoder	Non-linear	No	Learned pat-terns	Complex / image data	Moderate

Decision Guide

Default: **PCA**. Classification: **LDA**. Sparse/text: **SVD**. Visualization: **t-SNE/UMAP**. Complex non-linear: **Autoencoder**.

How to Choose the Right Extraction Method



- Feature extraction **transforms** original features into a new, smaller, mathematically derived set.
- **PCA** (unsupervised) is the most widely used method — maximizes variance, linear.
- **LDA** (supervised) maximizes class separation — ideal for classification preprocessing.
- **SVD** underpins PCA and excels on sparse data such as text (TF-IDF matrices).
- **t-SNE** and **UMAP** are non-linear methods best suited for visualization.
- **Autoencoders** learn non-linear compressions via neural networks.

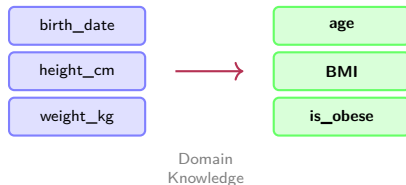
Key Takeaway

Feature extraction reduces dimensionality at the cost of interpretability — choose the method based on data type, task, and linearity of relationships.

- 1 Feature Extraction
 - Principal Component Analysis (PCA)
 - Linear Discriminant Analysis (LDA)
 - SVD, t-SNE, UMAP, and Autoencoders
- 2 Feature Construction
 - Mathematical and Polynomial Features
 - Temporal and Text Features
 - Aggregation, Binning, and Encoding
 - Interaction and Domain-Specific Features
- 3 Feature Selection
 - Filter Methods
 - Wrapper Methods
 - Embedded Methods
- 4 Principal Component Analysis (PCA): Introduction and The Mathematics
- 5 Factor Analysis
- 6 Discriminant Analysis
- 7 Kernel PCA
- 8 t-SNE (t-distributed Stochastic Neighbor Embedding)
- 9 Advanced Feature Engineering for Multivariate Data

Feature construction (feature engineering) is the process of **creating new features from existing data** using domain knowledge and creativity.

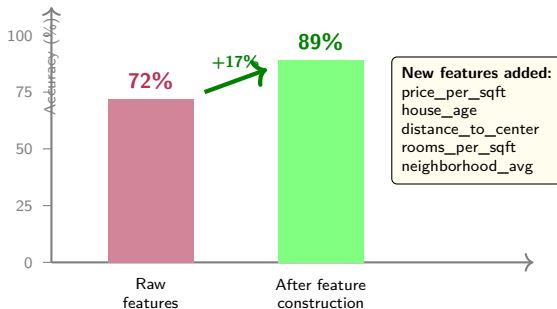
- Unlike extraction (mathematical transformation), construction uses **human insight**.
- Original features are **preserved**; new ones are **added**.
- Often the single most impactful step in ML pipelines.



Andrew Ng's Quote

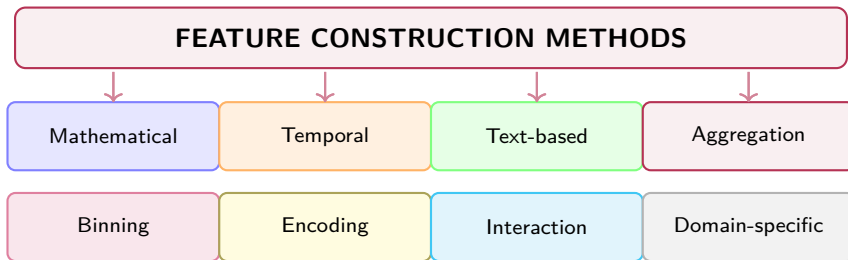
“Applied machine learning is basically feature engineering.” — Good features often matter more than complex models.

Impact of Feature Construction on Model Performance



Examples

House price prediction with only raw features (area, rooms, year) achieves 72%. Adding engineered features boosts accuracy to 89%.



Guiding Principle

The best features come from **understanding the problem domain**. Ask: "What information would a human expert use to make this decision?"

Creating new features using **arithmetic operations and mathematical functions** on existing

ones.

Type	Example	Use Case
Ratio	$\text{price_per_sqft} = \text{price} / \text{area}$	Normalize by size
Product	$\text{force} = \text{mass} \times \text{accel.}$	Physics-based
Difference	$\text{profit} = \text{revenue} - \text{cost}$	Business metric
Log	$\log_income = \log(\text{income})$	Reduce skewness
Square root	$\text{sqrt_area} = \sqrt{\text{area}}$	Reduce skewness
Polynomial	area^2 (quadratic)	Capture non-linearity
Reciprocal	$1 / \text{distance}$	Inverse relationship

Log Transform — Most Common

Highly skewed features (income, population, price) benefit from $\log(1 + x)$, which compresses the range and makes distributions more symmetric.

Creating mathematical features

```
import numpy as np

# Ratio features
df['price_per_sqft'] = df['price'] / df['area']
df['rooms_per_sqft'] = df['num_rooms'] / df['area']

# Log transformation (handles zeros with log1p)
df['log_income'] = np.log1p(df['income'])

# Polynomial features
df['area_squared'] = df['area'] ** 2

# Interaction term
df['area_x_rooms'] = df['area'] * df['num_rooms']

# Difference
df['net_profit'] = df['revenue'] - df['cost']
```

Tip

Always use `np.log1p(x)` instead of `np.log(x)` to safely handle zero values, since `log(0)` is undefined.

Automatically creates all polynomial combinations up to a given degree.



Features	Degree 2 output	Warning
2	5	Safe
5	20	Safe
10	65	Manageable
50	1,325	Large!
100	5,150	Very large!

Caution

Polynomial expansion grows rapidly! Use `interaction_only=True` to limit to interaction terms ($x_i \cdot x_j$) without self-powers (x_i^2).

Automated polynomial expansion with scikit-learn

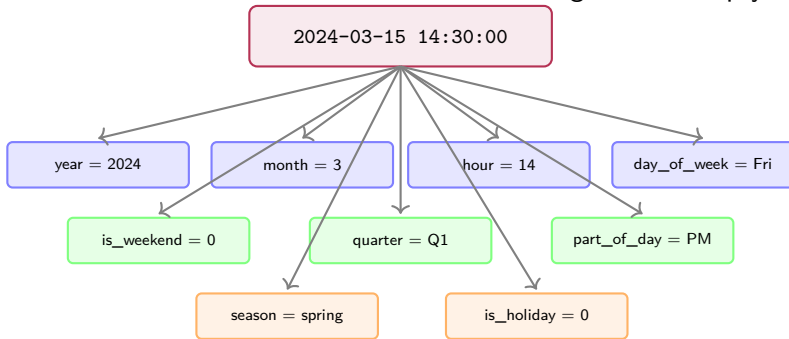
```
from sklearn.preprocessing import PolynomialFeatures

# Full degree-2 expansion
poly = PolynomialFeatures( degree=2,
interaction_only=False, include_bias=False)
X_poly = poly.fit_transform(X)

print(f"Original: {X.shape[1]} features")
print(f"Polynomial: {X_poly.shape[1]} features")
print(f"Names: {poly.get_feature_names_out()}")

# Interaction-only (no  $x^2$  terms)
poly_int = PolynomialFeatures( degree=2, interaction_only=True, include_bias=False)
X_int = poly_int.fit_transform(X)
```

Temporal data is one of the **richest sources** for feature construction. A single timestamp yields



many predictive features.

Extracting features from datetime

```
import pandas as pd, numpy as np

df['date'] = pd.to_datetime(df['timestamp'])

# Basic extraction
df['year'] = df['date'].dt.year
df['month'] = df['date'].dt.month
df['hour'] = df['date'].dt.hour
df['dow'] = df['date'].dt.dayofweek
df['is_weekend'] = df['dow'].isin([5,6]).astype(int)
df['quarter'] = df['date'].dt.quarter

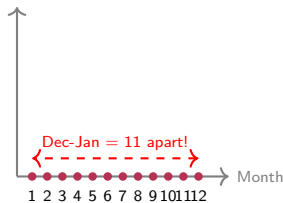
# Cyclical encoding (Dec->Jan smooth)
df['month_sin'] = np.sin(2*np.pi * df['month']/12)
df['month_cos'] = np.cos(2*np.pi * df['month']/12)

# Time differences
df['days_since'] = (
    pd.Timestamp.now() - df['last_purchase']).dt.days
```

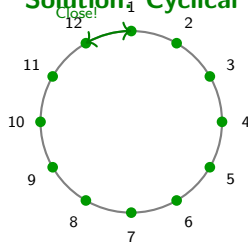
Why Cyclical Encoding?

Without it, month 12 (Dec) and month 1 (Jan) appear far apart numerically, but they are adjacent. Sine/cosine encoding captures this circular nature.

Problem: Raw Months



Solution: Cyclical



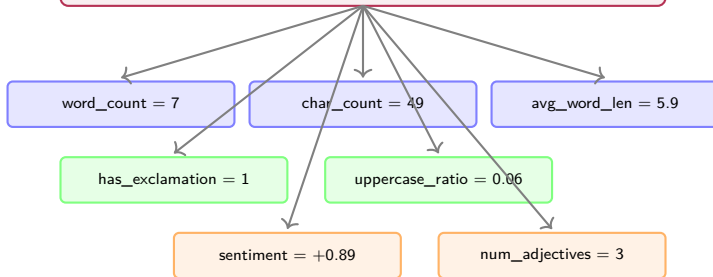
$$x_{\sin} = \sin\left(\frac{2\pi \cdot \text{month}}{12}\right) \quad x_{\cos} = \cos\left(\frac{2\pi \cdot \text{month}}{12}\right)$$

Same Principle Applies To

Hours of day (period = 24), days of week (period = 7), and any cyclical feature.

Extracting numeric features from **raw text data**.

"Amazing product! Great quality & fast delivery."



Beyond Basic Counts

Advanced text features include TF-IDF scores, sentiment polarity, named entity counts, readability scores, and embedding vectors.

Constructing features from text

```
import numpy as np
# Basic text features
df['word_count'] = df['text'].apply(lambda x: len(x.split()))
df['char_count'] = df['text'].apply(len)
df['avg_word_len'] = df['text'].apply(lambda x: np.mean([len(w) for w in x.split()]))
df['has_exclamation'] = (
df['text'].str.contains('!').astype(int))
df['uppercase_ratio'] = df['text'].apply(lambda x: sum(c.isupper() for c in x) / len(x))
```

Practical Tip

Combine these numeric text features with TF-IDF or word embeddings for stronger NLP models.

Creating features by computing **statistics within groups** of related records.

Raw Transactions

Cust	Amt	Cat
C01	50	Food
C01	200	Elec
C01	30	Food
C02	500	Elec

GROUP
BY →

Aggregated Features

Cust	Total	Avg	Txns	Cats
C01	280	93.3	3	2
C02	500	500	1	1

Common Aggregations

sum, mean, max, min, std, count, nunique — applied per group (customer, store, time window).

Group-based aggregation with pandas

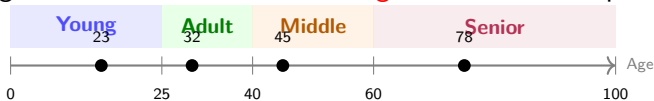
```
# Group-based aggregation
agg = df.groupby('customer_id').agg(
total_spend    = ('amount', 'sum'),
avg_spend      = ('amount', 'mean'),
max_spend      = ('amount', 'max'),
std_spend      = ('amount', 'std'),
num_txns       = ('amount', 'count'),
num_categories = ('category', 'nunique')) .reset_index()

# Merge back to original dataframe
df = df.merge(agg, on='customer_id', how='left')
```

Use Cases

Customer behavior profiling, per-store sales stats, per-patient medical summaries, rolling

Converting **continuous features** into **categorical bins** for simpler modeling.



Equal-Width Binning

Bins of equal range.

```
pd.cut(age, bins=[0,25,40,60,100])
```

Quantile Binning

Bins with equal number of samples.

```
pd.qcut(income, q=4)
```

Converting **categorical features** into **numeric** representations for ML models.

Label Encoding

Small $\rightarrow$ 0
Medium $\rightarrow$ 1
Large $\rightarrow$ 2
For ordinal data

One-Hot Encoding

Red $\rightarrow$ [1,0,0]
Blue $\rightarrow$ [0,1,0]
Green $\rightarrow$ [0,0,1]
For nominal data

Target Encoding

NYC $\rightarrow$ 0.72
LA $\rightarrow$ 0.58
SF $\rightarrow$ 0.81
High cardinality

Choosing the Right Encoding

Label: ordinal categories ($S < M < L$). **One-Hot:** nominal, low cardinality (< 10 categories).
Target/Frequency: high cardinality (cities, ZIP codes).

Four encoding strategies

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd

# 1. Label Encoding (ordinal)
le = LabelEncoder()
df['size_enc'] = le.fit_transform(df['size'])

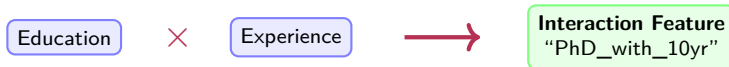
# 2. One-Hot Encoding (nominal)
df_ohe = pd.get_dummies(df, columns=['color'],
prefix='color')

# 3. Target Encoding
means = df.groupby('city')['target'].mean()
df['city_target'] = df['city'].map(means)

# 4. Frequency Encoding
freq = df['city'].value_counts(normalize=True)
```


Capturing the **combined effect of two or more features** that individually may not be predictive.

Education	Experience	Salary
PhD	10 years	High
PhD	2 years	Medium
High School	10 years	Medium
High School	2 years	Low



Key Insight

Neither education alone nor experience alone predicts salary well, but their **combination** does. Interaction features capture such joint effects.

The most powerful features come from **understanding the specific domain**.

Finance

$\text{debt_to_income} = \text{debt} / \text{income}$
 $\text{credit_util} = \text{balance} / \text{limit}$
 $\text{payment_ratio} = \text{on_time} / \text{total}$

Healthcare

$\text{BMI} = \text{weight} / \text{height}^2$
 age_at_diagnosis
 comorbidity_count

E-Commerce

cart_abandon_rate
 $\text{avg_session_duration}$
 $\text{recency} = \text{days_since_purchase}$

Real Estate

$\text{price_per_sqft} = \text{price} / \text{area}$
 $\text{distance_to_center}$
 $\text{house_age} = \text{year} - \text{built}$

Guiding Question

“What information would a **human expert** use to make this decision?” — Translate that expertise into computable features.

Category	Techniques	Example
Mathematical	Ratios, logs, polynomials, sqrt	price_per_sqft
Temporal	Date parts, time diffs, cyclical	is_weekend, hour_sin
Text-based	Word counts, sentiment, TF-IDF	avg_word_length
Aggregation	Group mean, sum, count, std	customer_total_spend
Binning	Equal-width, quantile	age_group = "Adult"
Encoding	One-hot, target, frequency	city_target_enc
Interaction	Products, concatenations	education × experience
Domain-specific	BMI, distances, ratios, derived	credit_utilization

Remember

Feature construction is often the **highest-impact** activity in an ML project. Good features can compensate for simpler models, but complex models cannot compensate for poor features.

Aspect	Selection	Extraction	Construction
What it does	Picks best originals	Transforms to new	Creates new manually
Originals kept?	Yes, as-is	No	Yes + new added
Interpretability	High	Low (abstract)	High (meaningful)
Domain knowledge	Not required	Not required	Essential
Automation	Fully automated	Fully automated	Mostly manual
Example	Keep “income”	PCA component 1	$BMI = wt/ht^2$
When to use	Too many features	Correlated features	Features not expressive

In Practice

Real-world projects use all three: **construct** domain features, **extract** compressed representations from high-dimensional data, then **select** the best features for the final model.

Feature Extraction

- PCA: unsupervised, max variance
- LDA: supervised, max class separation
- LDA components $\leq n_{\text{classes}} - 1$
- t-SNE/UMAP: visualization only
- Autoencoders: non-linear compression
- Always standardize before PCA

Feature Construction

- Domain knowledge is essential
- Log transform for skewed data
- Cyclical encoding for time
- Aggregation for grouped data
- One-hot: low cardinality
- Target encoding: high cardinality

Final Takeaway

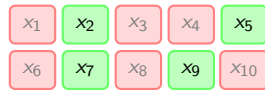
Feature extraction reduces dimensionality via transformation; feature construction enriches the feature space using domain insight. Together they form the backbone of effective feature engineering.

- 1 Feature Extraction
 - Principal Component Analysis (PCA)
 - Linear Discriminant Analysis (LDA)
 - SVD, t-SNE, UMAP, and Autoencoders
- 2 Feature Construction
 - Mathematical and Polynomial Features
 - Temporal and Text Features
 - Aggregation, Binning, and Encoding
 - Interaction and Domain-Specific Features
- 3 Feature Selection
 - Filter Methods
 - Wrapper Methods
 - Embedded Methods
- 4 Principal Component Analysis (PCA): Introduction and The Mathematics
- 5 Factor Analysis
- 6 Discriminant Analysis
- 7 Kernel PCA
- 8 t-SNE (t-distributed Stochastic Neighbor Embedding)
- 9 Advanced Feature Engineering for Multivariate Data

Feature selection is the process of **identifying and selecting the most relevant subset of input features** from a dataset that contribute most to predicting the target variable.

- Irrelevant and redundant features are discarded.
- Original features are retained (no transformation).
- Improves model accuracy, speed, and interpretability.

Original (10 features)



Selected (4 features)

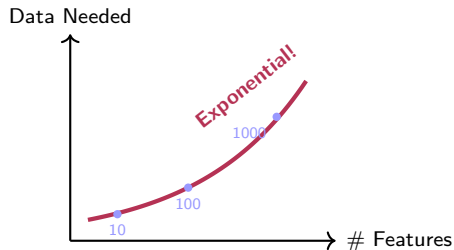


Important Distinction

Feature **selection** keeps original features. Feature **extraction** (e.g. PCA) transforms them into new ones.

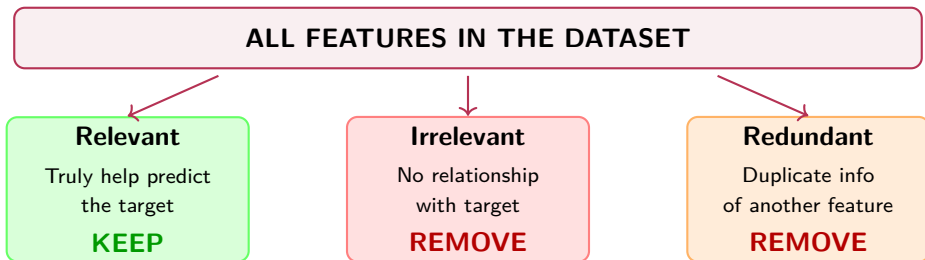
Problems of Too Many Features

- **Curse of Dimensionality** — data becomes sparse; models need exponentially more samples.
- **Overfitting** — irrelevant features act as noise; model memorizes instead of learning.
- **Multicollinearity** — redundant features destabilize coefficient estimates.



Benefits of Feature Selection

Better accuracy, faster training, improved interpretability, lower storage costs, and reduced multicollinearity.



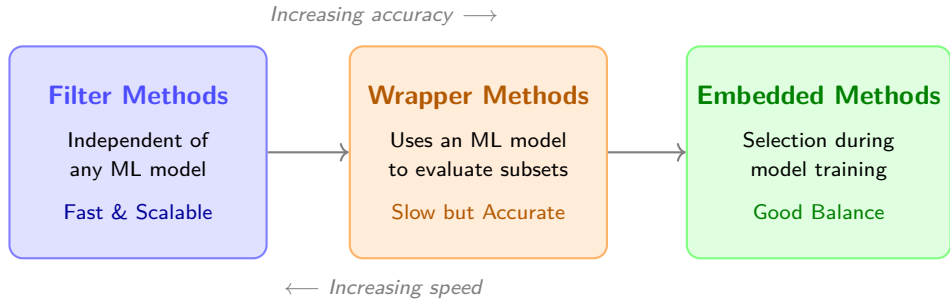
Strongly Relevant

Removing it *always* degrades performance.

Weakly Relevant

Useful only in combination with other features.

Three Main Approaches to Feature Selection



Practical Insight

Filter for quick screening, Embedded for general use, Wrapper for fine-tuning on small datasets.

Filter methods evaluate each feature **independently** using statistical measures, **before** any model is trained.



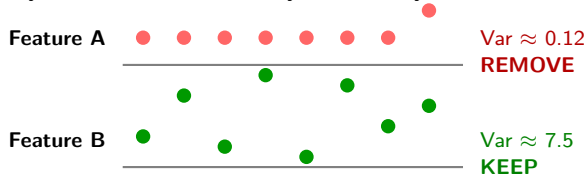
Strengths

- Very fast and scalable
- Model-independent
- Good for initial screening

Limitations

- Ignores feature interactions
- May select redundant features
- Only pairwise (feature $\leftrightarrow$ target)

Idea: Remove features with very low variance — they are nearly constant and carry almost no information.



When to Use

As a first quick pass to eliminate obviously useless (near-constant) features before applying other methods.

VarianceThreshold in scikit-learn

```
from sklearn.feature_selection import VarianceThreshold

# Remove features with variance < 0.1
selector = VarianceThreshold(threshold=0.1)
X_selected = selector.fit_transform(X)

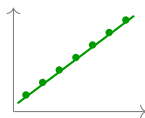
print("Original features:", X.shape[1])
print("Selected features:", X_selected.shape[1])
```

Note

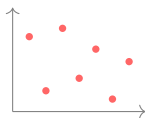
For binary features, a variance threshold of $p(1 - p)$ at $p = 0.8$ removes features that are 80% or more constant: $\text{threshold} = 0.8 \times 0.2 = 0.16$.

Measures the **linear** relationship between a numeric feature and a numeric target.

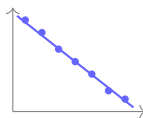
$$r_{xy} = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}} \quad r \in [-1, +1]$$



$r = +0.95$
KEEP



$r = 0.03$
REMOVE



$r = -0.92$
KEEP

Feature–Feature Correlation for Redundancy Removal

If two features have $|r| > 0.95$ with each other, one is redundant and should be dropped.

Feature-Target and Feature-Feature Correlation

```
import pandas as pd, numpy as np

# Feature-Target correlation
corr = df.corr()['target'].drop('target').abs()
selected = corr[corr > 0.3].index.tolist()

# Feature-Feature: remove redundancy (r > 0.95)
corr_matrix = df.corr().abs()
upper = corr_matrix.where(
    np.triu(np.ones(corr_matrix.shape), k=1).astype(
        bool))
to_drop = [c for c in upper.columns
    if any(upper[c] > 0.95)]
df_reduced = df.drop(columns=to_drop)
```

Feature	r with Target	Action
income	0.85	KEEP
credit_score	0.72	KEEP
age	0.45	KEEP
zip_code	0.03	REMOVE
random_id	0.01	REMOVE

Use case: Categorical features $\rightarrow$ Categorical target.

Tests whether a feature is **independent** of the target. High $\chi^2 \Rightarrow$ feature is relevant.

$$\chi^2 = \sum \frac{(O_{ij} - E_{ij})^2}{E_{ij}} \quad \text{where } E_{ij} = \frac{(\text{Row Total}_i)(\text{Col Total}_j)}{N}$$

	Target = Yes	Target = No	Total
Feature = A	40 (Exp: 30)	10 (Exp: 20)	50
Feature = B	20 (Exp: 30)	30 (Exp: 20)	50
Total	60	40	100

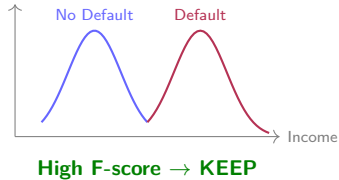
$$\chi^2 = \frac{(40-30)^2}{30} + \frac{(10-20)^2}{20} + \frac{(20-30)^2}{30} + \frac{(30-20)^2}{20} = 16.67 \rightarrow \text{KEEP}$$

Rule

High χ^2 value $\Rightarrow$ feature is NOT independent of target $\Rightarrow$ **keep the feature.**

Use case: Numeric features → Categorical target.

Tests whether the **means** of a numeric feature differ significantly across target classes.



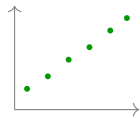
$$F = \frac{\text{Between-group variance}}{\text{Within-group variance}}$$

Interpretation

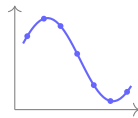
Large F means feature values differ strongly across classes — the feature is discriminative.

Measures the **total dependency** (including non-linear) between a feature and the target.

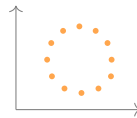
$$MI(X; Y) = \sum_x \sum_y p(x, y) \log \frac{p(x, y)}{p(x) p(y)}$$



Correlation: ✓
MI: ✓



Correlation: ✗
MI: ✓



Correlation: ✗
MI: ✓

Examples

MI = 0 means complete independence. Unlike Pearson correlation, MI captures curved, circular, and all non-linear patterns.

Using SelectKBest with different score functions

```
from sklearn.feature_selection import (  
    SelectKBest, chi2, f_classif,  
    mutual_info_classif )  
  
# Chi-Square (categorical features)  
sel_chi = SelectKBest(chi2, k=5)  
X_chi = sel_chi.fit_transform(X, y)  
  
# ANOVA F-test (numeric features)  
sel_f = SelectKBest(f_classif, k=10)  
X_f = sel_f.fit_transform(X, y)  
  
# Mutual Information (any feature type)  
sel_mi = SelectKBest(mutual_info_classif, k=10)  
X_mi = sel_mi.fit_transform(X, y)
```

Choosing the Right Score Function

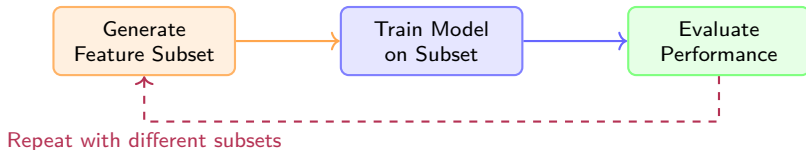
Match the score function to your feature type and target type for meaningful results.

Method	Feature Type	Target Type	Captures	Speed
Variance Threshold	Any	None needed	Low-variance features	Very Fast
Pearson Correlation	Numeric	Numeric	Linear relationships	Fast
Chi-Square (χ^2)	Categorical	Categorical	Association	Fast
ANOVA F-test	Numeric	Categorical	Mean differences	Fast
Mutual Information	Any	Any	Any dependency (incl. non-linear)	Moderate

Key Point

Filter methods are ideal for **initial screening** on large datasets. Follow up with embedded or wrapper methods for refined selection.

Wrapper methods use an **actual ML model** to evaluate different feature subsets. They “wrap” around the model and test subset performance.



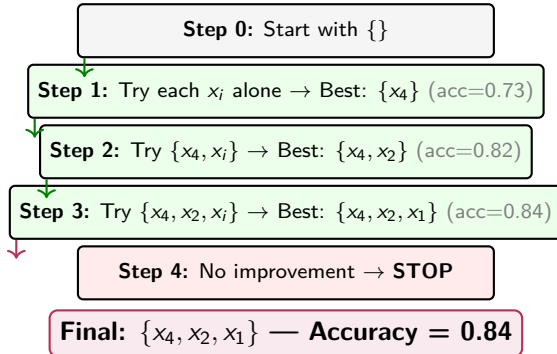
Strengths

- Considers feature interactions
- Usually more accurate selection
- Finds jointly useful features

Limitations

- Very computationally expensive
- Risk of overfitting
- Tied to one specific model

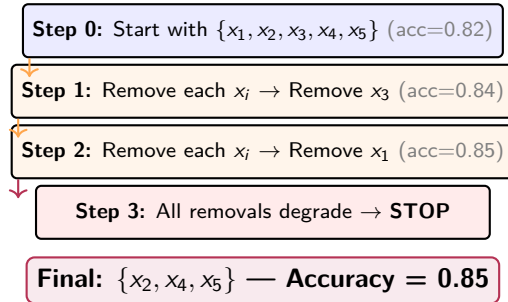
Strategy: Start with **no features**, add the best one at each step.



Characteristic

Greedy approach — fast but may miss globally optimal combinations.

Strategy: Start with **all features**, remove the worst one at each step.



Comparison

Forward starts empty and builds up; Backward starts full and trims down. Backward is more expensive but may preserve important interactions.

Strategy: Train a model, rank features by importance, remove the weakest, **repeat**.

Round 1: 10 features → remove x_3 (weakest)



Round 2: 9 features → remove x_8 (weakest)



Round 3: 8 features → remove x_1 (weakest)



⋮

Final: 5 features — target count reached

RFECV Variant

RFECV uses **cross-validation** to automatically determine the optimal number of features, rather than requiring a fixed count.

Forward/Backward Selection & RFE

```
from sklearn.feature_selection import
    (
        SequentialFeatureSelector, RFE, RFECV
    )
from sklearn.ensemble import
    RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100)

# Forward Selection
sfs = SequentialFeatureSelector(
    model, n_features_to_select=5,
    direction='forward', cv=5)
sfs.fit(X, y)
```

Forward/Backward Selection & RFE

```
# Backward Elimination
sbs = SequentialFeatureSelector(
    model, n_features_to_select=5,
    direction='backward', cv=5)
sbs.fit(X, y)

# RFECV - auto-selects optimal number
rfecv = RFECV(estimator=model, step
    =1,
    cv=5, scoring='accuracy')
rfecv.fit(X, y)
print("Optimal:", rfecv.n_features_)
```

Strategy: Try **all possible** feature subsets and pick the best one.

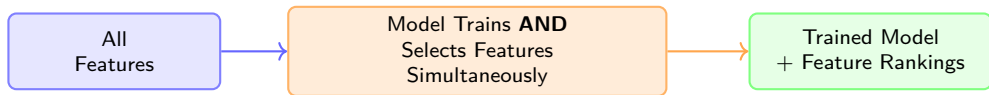
$$\text{Total subsets} = 2^n - 1$$

# Features (n)	Subsets ($2^n - 1$)	Feasibility
4	15	Easy
10	1,023	Feasible
20	1,048,575	Slow
50	$\approx 1.13 \times 10^{15}$	Impossible
100	$\approx 1.27 \times 10^{30}$	Utterly impossible

Conclusion

Exhaustive search is **almost never used in practice** due to exponential computational cost. Use greedy approaches (Forward, Backward, RFE) instead.

Feature selection is **built into the model training process** itself. The model simultaneously learns the task and identifies important features.



Key Methods

LASSO (L1 regularization), Ridge (L2), Elastic Net (L1+L2), and Tree-based feature importance.

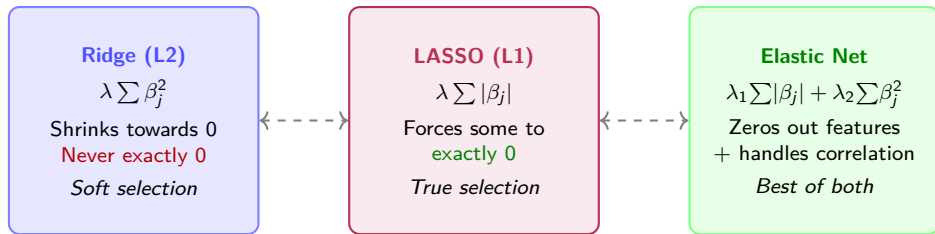
LASSO adds a penalty equal to the **absolute value** of coefficients, forcing some to become **exactly zero**.

$$\mathcal{L}_{\text{LASSO}} = \underbrace{\sum_i (y_i - \hat{y}_i)^2}_{\text{MSE}} + \lambda \underbrace{\sum_j |\beta_j|}_{\text{L1 Penalty}}$$

Feature	Without LASSO	With LASSO	Status
income	0.85	0.72	Kept (shrunk)
age	0.42	0.28	Kept (shrunk)
zip_code	0.08	0.00	REMOVED
credit	0.65	0.51	Kept (shrunk)
shoe_size	0.03	0.00	REMOVED
debt	0.55	0.40	Kept (shrunk)

Key Property

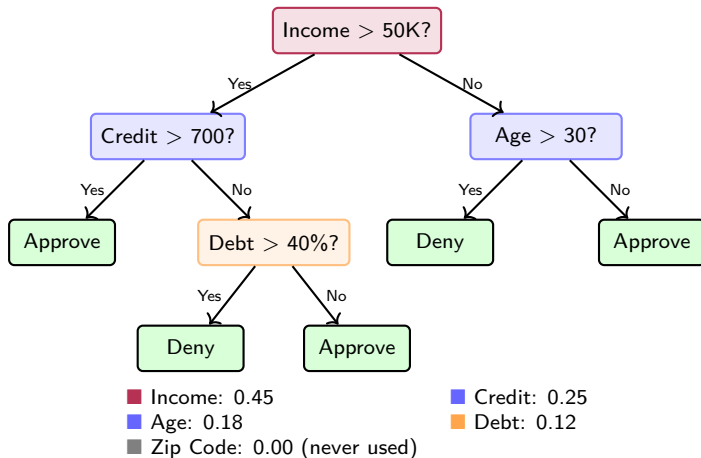
LASSO performs **automatic feature selection** by zeroing out irrelevant coefficients.



When to Use Each

LASSO for sparse feature selection, **Ridge** when all features matter but need shrinkage, **Elastic Net** when features are correlated and you need selection.

Decision trees and ensembles naturally rank features by how useful they are for **splitting nodes**.



Random Forest Feature Importance + SelectFromModel

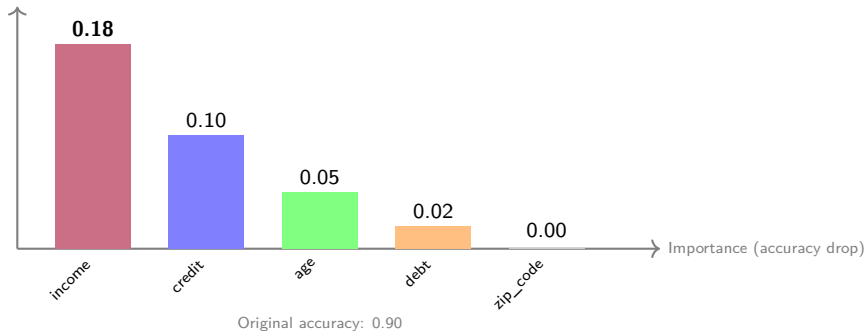
```
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel
import pandas as pd

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X, y)

# Feature importance ranking
imp = pd.DataFrame({ 'Feature': feature_names,
'Importance': rf.feature_importances_
}).sort_values('Importance', ascending=False)
print(imp)

# Select features above mean importance
selector = SelectFromModel(rf, threshold='mean')
selector.fit(X, y)
X_selected = selector.transform(X)
print(f"Selected {X_selected.shape[1]} of " f"{X.shape[1]} features")
```

Idea: After training, **randomly shuffle** one feature's values and measure the drop in performance. Large drop = important feature.



Advantage

Model-agnostic - works with any trained model. Measures real predictive contribution rather than proxy statistics.

Embedded Methods Implementation

```
from sklearn.linear_model import LassoCV, ElasticNetCV
from sklearn.inspection import permutation_importance

# LASSO with cross-validated lambda
lasso = LassoCV(cv=5, random_state=42)
lasso.fit(X, y)
selected = [f for f, c in zip(feature_names,
lasso.coef_) if c != 0]

# Elastic Net
enet = ElasticNetCV(l1_ratio=[0.1,0.5,0.7,0.9,1.0], cv=5) enet.fit(X, y)

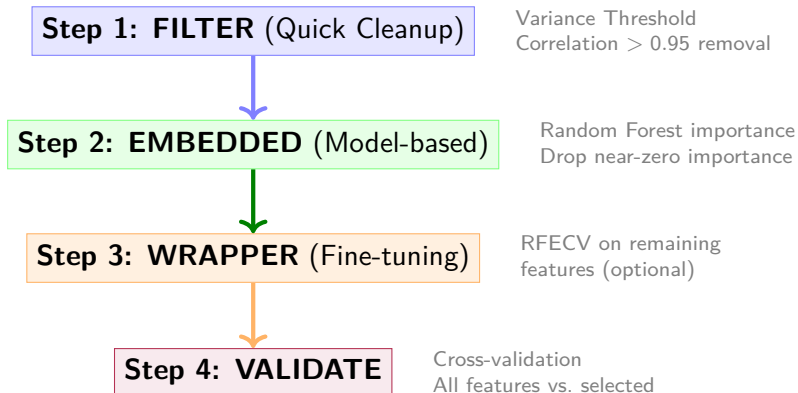
# Permutation Importance (model-agnostic)
result = permutation_importance(
rf, X_test, y_test,
n_repeats=10, random_state=42)
print(result.importances_mean)
```

Complete Comparison: Filter vs Wrapper vs Embedded

Aspect	Filter	Wrapper	Embedded
When it runs	Before training	Around training	During training
Speed	Very Fast	Very Slow	Moderate
Interactions	Ignores	Captures	Captures
Overfitting risk	Low	High	Moderate
Model dependency	None	Full	Partial
Scalability	Millions of features	< hundreds	Thousands
Accuracy	Moderate	High	High
Best for	Initial screening	Final tuning	General purpose

Practical Guideline

Use filters first to reduce dimensionality, then embedded methods for robust selection, and optionally wrappers for final fine-tuning on smaller feature sets.



End-to-End Feature Selection Pipeline

```
from sklearn.feature_selection import
    VarianceThreshold, RFECV
from sklearn.ensemble import
    RandomForestClassifier
from sklearn.model_selection import
    cross_val_score
import numpy as np, pandas as pd

# Step 1: Filter - remove low variance
vt = VarianceThreshold(threshold=0.01)
X_v = vt.fit_transform(X_train)

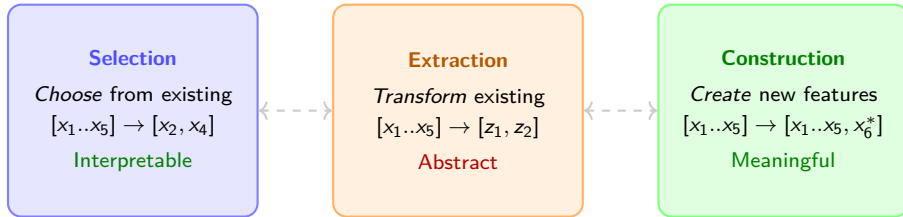
# Step 2: Filter - remove correlated (r > 0.95)
corr = pd.DataFrame(X_v).corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k
    =1).astype(bool))
drop = [c for c in upper.columns if any(upper[c]
    > 0.95)]
X_c = pd.DataFrame(X_v).drop(columns=drop)
```

End-to-End Feature Selection Pipeline

```
# Step 3: Embedded - RF importance > mean
rf = RandomForestClassifier(n_estimators=100,
    random_state=42)
rf.fit(X_c, y_train)
mask = rf.feature_importances_ > np.mean(rf.
    feature_importances_)
X_rf = X_c.loc[:, mask]

# Step 4: Wrapper - RFECV
rfecv = RFECV(rf, step=1, cv=5, scoring='accuracy
    ')
rfecv.fit(X_rf, y_train)

# Step 5: Compare
s_all = cross_val_score(rf, X_train, y_train, cv
    =5).mean()
s_sel = cross_val_score(rf, X_rf.loc[:,rfecv.
    support_], y_train, cv=5).mean()
print(f"All: {s_all:.4f} | Selected: {s_sel:.4f}")
```



When to Use Which

Selection: too many features. **Extraction:** correlated/redundant features. **Construction:** raw features not expressive enough.

Conceptual

- Feature **selection** $\neq$ feature **extraction** (PCA)
- Three types: Filter $\rightarrow$ Wrapper $\rightarrow$ Embedded
- Curse of dimensionality: more features $\Rightarrow$ exponentially more data needed

Technical

- LASSO (L1): sets $\beta = 0 \Rightarrow$ true selection
- Ridge (L2): shrinks only $\Rightarrow$ **not** selection
- Chi-Square: Cat $\rightarrow$ Cat
- ANOVA: Num $\rightarrow$ Cat
- MI: Any $\rightarrow$ Any

Workflow Tips

Forward selection starts **empty** and adds; Backward elimination starts **full** and removes. RFE trains, ranks, removes weakest, and repeats. Embedded methods balance filter speed with wrapper accuracy.

- Feature selection identifies and retains the most relevant features while discarding irrelevant and redundant ones.
- **Filter methods** are fast, model-independent, and suitable for initial screening using statistical tests.
- **Wrapper methods** evaluate actual model performance on feature subsets but are computationally expensive.
- **Embedded methods** integrate selection into training, offering a practical balance of speed and accuracy.
- A layered pipeline (Filter → Embedded → Wrapper) provides the most robust results.

Takeaway

Good feature selection improves accuracy, reduces overfitting, speeds up training, and enhances model interpretability.

- 1 Feature Extraction
 - Principal Component Analysis (PCA)
 - Linear Discriminant Analysis (LDA)
 - SVD, t-SNE, UMAP, and Autoencoders
- 2 Feature Construction
 - Mathematical and Polynomial Features
 - Temporal and Text Features
 - Aggregation, Binning, and Encoding
 - Interaction and Domain-Specific Features
- 3 Feature Selection
 - Filter Methods
 - Wrapper Methods
 - Embedded Methods
- 4 Principal Component Analysis (PCA): Introduction and The Mathematics
- 5 Factor Analysis
- 6 Discriminant Analysis
- 7 Kernel PCA
- 8 t-SNE (t-distributed Stochastic Neighbor Embedding)
- 9 Advanced Feature Engineering for Multivariate Data

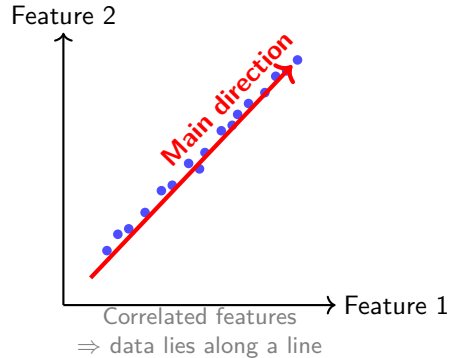
Every dataset lives in a **space** defined by its features (columns).

The Curse of Dimensionality

- A dataset with p features lives in $\mathbb{R}^p$
- As p grows: distances lose meaning, models overfit, computation explodes
- Many features are **redundant** or **correlated**

Example

A housing dataset with *sq. footage*, *rooms*, *bathrooms*, *garage size*, *lot size* has $p = 5$, but a single “**overall size**” factor explains most variation.



Definition

Principal Component Analysis (PCA) is an **unsupervised, linear** dimensionality reduction technique that transforms data into a new coordinate system where the axes (**principal components**) are:

- 1 Ordered by the amount of **variance** they capture
- 2 Mutually **orthogonal** (uncorrelated)
- 3 Linear combinations of the original features

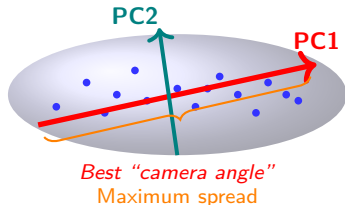
Key Properties

- **Unsupervised:** No labels needed
- **Linear:** Finds straight axes only
- **Lossless** if all PCs kept
- **Lossy compression** when $k < p$

Applications

- Face recognition (Eigenfaces)
- Gene expression analysis
- Image compression
- Noise reduction
- Data visualization (high-D $\rightarrow$ 2D/3D)

Imagine capturing a 3D sculpture in a single 2D photograph.



The Idea

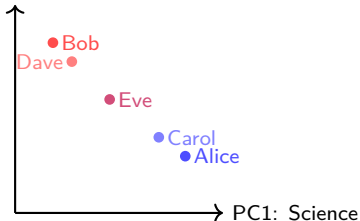
- Walk around the data "sculpture"
- Find the angle showing **maximum spread**
- That's **PC1**
- Next best **perpendicular** angle → **PC2**
- Repeat for each dimension

Goal: Keep the top k "views" that capture $\geq 90-95\%$ of total information.

Scenario: 5 subjects, but only 2 **hidden factors** drive the scores.

Student	Ma	Ph	Ch	En	Hi
Alice	95	92	90	60	55
Bob	40	45	42	92	88
Carol	88	85	80	65	60
Dave	50	48	52	85	82
Eve	70	68	65	75	72

PC2: Humanities

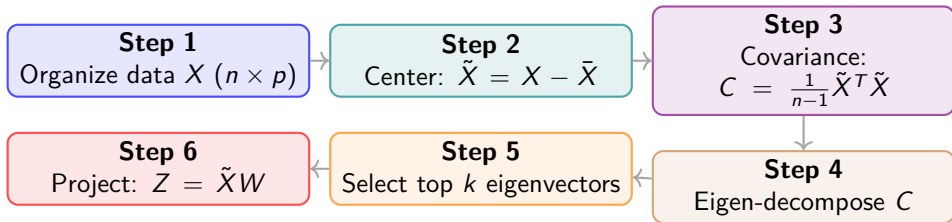


What PCA Discovers

- **PC1:** “Science aptitude” — loads on Math, Physics, Chemistry
- **PC2:** “Humanities aptitude” — loads on English, History
- 5 features → 2 meaningful components

Key Insight

PCA finds **hidden structure** automatically. Correlated variables get *compressed* into single components without any human labeling.



Inputs

- Data matrix $X \in \mathbb{R}^{n \times p}$
- Number of components $k \leq p$

Outputs

- Transformed data $Z \in \mathbb{R}^{n \times k}$
- Explained variance per component
- Loading vectors (eigenvectors)

Mean (Center of Data)

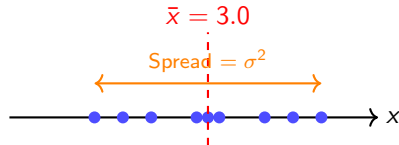
$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

PCA **centers** data by subtracting the mean so the cloud sits at the origin.

Variance (Spread)

$$\sigma_x^2 = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2$$

Measures how **spread out** a feature is. PCA seeks directions that **maximize** this.



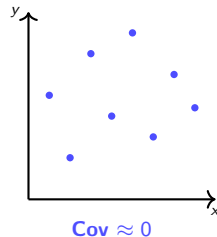
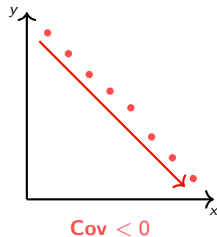
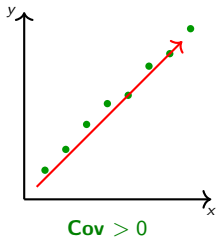
Standard Deviation

$$\sigma_x = \sqrt{\sigma_x^2}$$

Large σ $\Rightarrow$ feature carries more information (more variation to explain).

Definition

Covariance measures how two features **co-vary**:
$$\text{Cov}(x, y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$



Why Covariance Matters for PCA

High covariance between features $\Rightarrow$ redundancy $\Rightarrow$ PCA can **compress** them. If all covariances were zero, each original axis would already be a principal component.

Definition

For p features, the covariance matrix C is a **symmetric** $p \times p$ matrix:

$$C = \frac{1}{n-1} \tilde{X}^T \tilde{X} = \begin{bmatrix} \sigma_1^2 & \text{Cov}(x_1, x_2) & \cdots & \text{Cov}(x_1, x_p) \\ \text{Cov}(x_2, x_1) & \sigma_2^2 & \cdots & \text{Cov}(x_2, x_p) \\ \vdots & \vdots & \ddots & \vdots \\ \text{Cov}(x_p, x_1) & \text{Cov}(x_p, x_2) & \cdots & \sigma_p^2 \end{bmatrix}$$

Key Properties

- **Symmetric:** $C = C^T$
- **Diagonal** entries: variances
- **Off-diagonal:** pairwise covariances
- **Positive semi-definite:** all eigenvalues ≥ 0

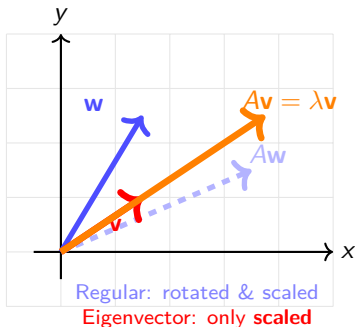
Example ($p = 3$)

$$C = \begin{bmatrix} 4.2 & 3.1 & 0.2 \\ 3.1 & 3.8 & -0.1 \\ 0.2 & -0.1 & 1.5 \end{bmatrix}$$

Features 1 & 2 are highly correlated (3.1), but feature 3 is nearly independent.

Definition

For a square matrix A , vector $\mathbf{v} \neq \mathbf{0}$ and scalar λ satisfying $A\mathbf{v} = \lambda\mathbf{v}$ are an **eigenvector** and **eigenvalue**.



For PCA

- **Eigenvectors** of C = **directions** of PCs
- **Eigenvalues** of C = **variance** along each PC
- Largest $\lambda \rightarrow$ PC1, next $\rightarrow$ PC2, ...

Finding Eigenvalues

Solve the **characteristic equation**:

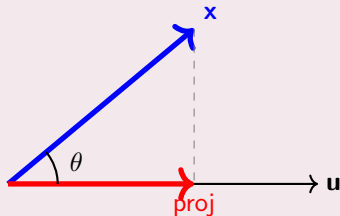
$$\det(A - \lambda I) = 0$$

Then back-substitute to find each $\mathbf{v}$.

Dot Product & Projection

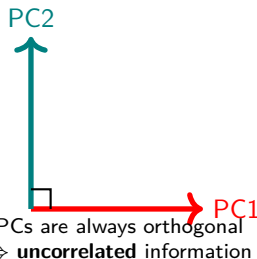
Projection of $\mathbf{x}$ onto unit vector $\mathbf{u}$:

$$\text{proj} = \mathbf{x} \cdot \mathbf{u} = \sum_i x_i u_i$$



Orthogonality

Two vectors are **orthogonal** if $\mathbf{u} \cdot \mathbf{v} = 0$.



Why Orthogonal?

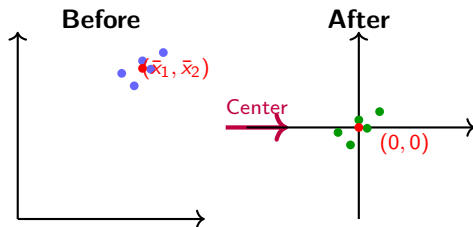
Guaranteed by the **Spectral Theorem**: real symmetric matrix has orthogonal eigenvectors.

Data Matrix

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}_{n \times p} \quad \tilde{x}_{ij} = x_{ij} - \bar{x}_j$$

Why Center?

Centering ensures PCA finds directions of **variance**, not directions biased by where the data sits in space. After centering each column has mean = 0.



Step 2: Covariance Matrix

$$C = \frac{1}{n-1} \tilde{X}^T \tilde{X} \in \mathbb{R}^{p \times p}$$

Step 3: The Optimization Problem

Find unit vector $\mathbf{u}$ maximizing **projected variance**:

$$\max_{\mathbf{u}} \mathbf{u}^T C \mathbf{u} \quad \text{subject to} \quad \|\mathbf{u}\| = 1$$

Derivation of the Objective

Let $\mathbf{z} = \tilde{X}\mathbf{u}$ be projected coordinates. Then:

$$\text{Var}(\mathbf{z}) = \frac{1}{n-1} \mathbf{z}^T \mathbf{z} = \frac{1}{n-1} (\tilde{X}\mathbf{u})^T (\tilde{X}\mathbf{u}) = \mathbf{u}^T \underbrace{\left(\frac{\tilde{X}^T \tilde{X}}{n-1} \right)}_C \mathbf{u} = \mathbf{u}^T C \mathbf{u}$$

Lagrangian

$$\mathcal{L}(\mathbf{u}, \lambda) = \mathbf{u}^T C \mathbf{u} - \lambda (\mathbf{u}^T \mathbf{u} - 1)$$

Derivative $\rightarrow$ Eigenvalue Equation

$$\frac{\partial \mathcal{L}}{\partial \mathbf{u}} = 2C\mathbf{u} - 2\lambda\mathbf{u} = 0$$

$$C\mathbf{u} = \lambda\mathbf{u}$$

Variance along $\mathbf{u}$:

$$\mathbf{u}^T C \mathbf{u} = \mathbf{u}^T \lambda \mathbf{u} = \lambda$$

The eigenvalue is the variance!

Therefore

- 1 Compute all $(\lambda_i, \mathbf{u}_i)$ of C
- 2 Sort: $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$
- 3 **PC1** = $\mathbf{u}_1$ (largest λ)
- 4 **PC2** = $\mathbf{u}_2$ (second largest)

Orthogonality

Since C is symmetric, the Spectral Theorem guarantees eigenvectors are **automatically orthogonal**.

Step 5: Select Top k Components

Projection matrix: $W = [\mathbf{u}_1 \quad \mathbf{u}_2 \quad \cdots \quad \mathbf{u}_k] \in \mathbb{R}^{p \times k}$ Choose k so that:

$$\frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^p \lambda_j} \geq 0.90-0.95$$

Step 6: Transform

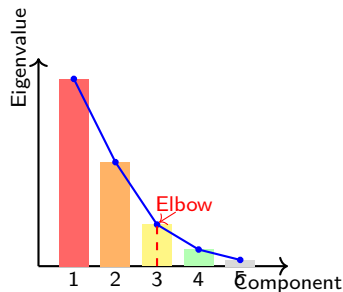
$$Z = \tilde{X} W \in \mathbb{R}^{n \times k}$$

Reconstruction

$$\hat{X} = Z W^T + \bar{X}$$

$$\text{Error} = \sum_{i=k+1}^p \lambda_i$$

Scree Plot (choosing k)



Pick k at the “elbow” where returns diminish sharply.

Explained Variance Ratio

$$\text{EVR}_i = \frac{\lambda_i}{\sum_{j=1}^p \lambda_j} \quad \text{Cumulative: } \text{CEVR}_k = \sum_{i=1}^k \text{EVR}_i$$

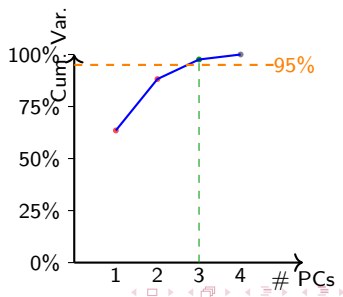
Numerical Example

Eigenvalues: $\lambda_1 = 5.4$, $\lambda_2 = 2.1$, $\lambda_3 = 0.8$, $\lambda_4 = 0.2$. Total

PC	EVR	Cumulative
1	63.5%	63.5%
2	24.7%	88.2%
3	9.4%	97.6%
4	2.4%	100%

⇒ Keep **3 PCs** for > 95% retention.

Cumulative Variance Plot



Covariance Matrix PCA

Use $C = \frac{1}{n-1} \tilde{X}^T \tilde{X}$

- Features in the **same units/scale**
- Larger-variance features **dominate**

Danger!

If feature 1 is in meters and feature 2 in mm, feature 2's variance is $10^6 \times$ larger — PCA is dominated by *scale*, not information.

Correlation Matrix PCA

Standardize first: $z_{ij} = \frac{x_{ij} - \bar{x}_j}{\sigma_j}$, then C becomes correlation matrix R :

$$R_{jk} = \frac{\text{Cov}(x_j, x_k)}{\sigma_j \sigma_k}$$

- Features on **equal footing**
- Use for **different scales/units**
- Most common in practice

Rule of Thumb

Different units? $\Rightarrow$ Correlation matrix.

Same units, variance matters? $\Rightarrow$ Covariance.

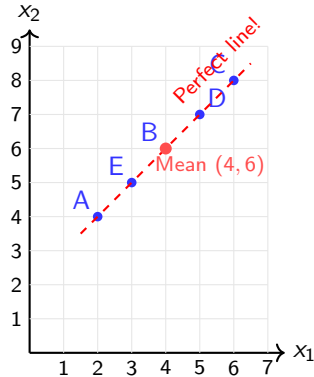
Solved Example 1: Setup (Perfect Correlation)

Problem

Apply PCA to this 2D dataset and reduce to 1D.

Pt	x_1	x_2
A	2	4
B	4	6
C	6	8
D	5	7
E	3	5

$n = 5, p = 2$



Solved Example 1: Centering and Covariance Matrix

Step 1: Means

$$\bar{x}_1 = \frac{2 + 4 + 6 + 5 + 3}{5} = 4$$

$$\bar{x}_2 = \frac{4 + 6 + 8 + 7 + 5}{5} = 6$$

Step 2: Center

$$\tilde{X} = \begin{bmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 2 \\ 1 & 1 \\ -1 & -1 \end{bmatrix}$$

Step 3: Covariance Matrix

$\tilde{X}^T \tilde{X}$:

$$C_{11} = \frac{4 + 0 + 4 + 1 + 1}{4} = 2.5$$

$$C_{12} = \frac{4 + 0 + 4 + 1 + 1}{4} = 2.5$$

$$C_{22} = \frac{4 + 0 + 4 + 1 + 1}{4} = 2.5$$

$$C = \begin{bmatrix} 2.5 & 2.5 \\ 2.5 & 2.5 \end{bmatrix}$$

Observation

Cov = Var $\Rightarrow$ **perfect correlation** ($r = 1$).

Characteristic Equation: $\det(C - \lambda I) = 0$

$$\det \begin{bmatrix} 2.5 - \lambda & 2.5 \\ 2.5 & 2.5 - \lambda \end{bmatrix} = 0$$

Expanding

$$\begin{aligned} (2.5 - \lambda)^2 - (2.5)^2 &= 0 \\ 6.25 - 5\lambda + \lambda^2 - 6.25 &= 0 \\ \lambda^2 - 5\lambda &= 0 \\ \lambda(\lambda - 5) &= 0 \end{aligned}$$

$$\lambda_1 = 5, \quad \lambda_2 = 0$$

Interpretation

- $\lambda_1 = 5$: **All** variance along PC1
- $\lambda_2 = 0$: **Zero** variance $\perp$ to PC1
- Data is truly **1-dimensional**

Verification

$$\lambda_1 + \lambda_2 = 5 = \text{tr}(C) = 2.5 + 2.5 \quad \checkmark$$

For $\lambda_1 = 5$

$(C - 5I)\mathbf{u} = \mathbf{0}$:

$$\begin{bmatrix} -2.5 & 2.5 \\ 2.5 & -2.5 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$u_1 = u_2$. Normalize: $2u_1^2 = 1$

$$\mathbf{u}_1 = \begin{bmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{bmatrix} \approx \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix}$$

For $\lambda_2 = 0$

$(C - 0I)\mathbf{u} = \mathbf{0}$:

$$\begin{bmatrix} 2.5 & 2.5 \\ 2.5 & 2.5 \end{bmatrix} \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$u_2 = -u_1$

$$\mathbf{u}_2 = \begin{bmatrix} 1/\sqrt{2} \\ -1/\sqrt{2} \end{bmatrix}$$

Interpretation

PC1 at 45 — both features contribute equally. It captures the “average” direction.

Verify Orthogonality

$$\mathbf{u}_1 \cdot \mathbf{u}_2 = \frac{1}{2} - \frac{1}{2} = 0 \quad \checkmark$$

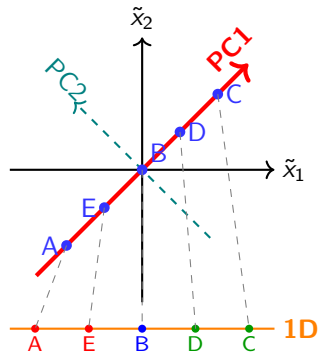
Solved Example 1: Projection and Result

Projection: $z_i = \tilde{\mathbf{x}}_i \cdot \mathbf{u}_1 = \frac{1}{\sqrt{2}}(\tilde{x}_{i1} + \tilde{x}_{i2})$

Pt	$\tilde{x}_1$	$\tilde{x}_2$	z
A	-2	-2	$-2\sqrt{2} \approx -2.83$
B	0	0	0
C	2	2	$2\sqrt{2} \approx 2.83$
D	1	1	$\sqrt{2} \approx 1.41$
E	-1	-1	$-\sqrt{2} \approx -1.41$

Explained Variance

$EVR_1 = \frac{5}{5+0} = 100\%$
2D $\rightarrow$ 1D with zero loss!



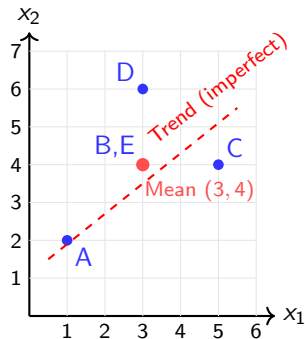
Solved Example 2: Setup (Partial Correlation)

Problem

Apply PCA where features are **not perfectly correlated**. Compute information loss.

Pt	x_1	x_2
A	1	2
B	3	4
C	5	4
D	3	6
E	3	4

$$n = 5, p = 2$$



Solved Example 2: Centering and Covariance

Step 1: Means

$$\bar{x}_1 = \frac{1+3+5+3+3}{5} = 3$$

$$\bar{x}_2 = \frac{2+4+4+6+4}{5} = 4$$

Step 2: Center

$$\tilde{X} = \begin{bmatrix} -2 & -2 \\ 0 & 0 \\ 2 & 0 \\ 0 & 2 \\ 0 & 0 \end{bmatrix}$$

Step 3: Covariance Matrix

$$\begin{aligned} \tilde{X}^T \tilde{X} &= \begin{bmatrix} 4+0+4+0+0 & 4+0+0+0+0 \\ 4+0+0+0+0 & 4+0+0+4+0 \end{bmatrix} \\ &= \begin{bmatrix} 8 & 4 \\ 4 & 8 \end{bmatrix} \end{aligned}$$

$$C = \frac{1}{4} \begin{bmatrix} 8 & 4 \\ 4 & 8 \end{bmatrix} = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$$

Observation

$$r = \frac{\text{Cov}}{\sigma_1 \sigma_2} = \frac{1}{\sqrt{2} \cdot \sqrt{2}} = 0.5 \text{ — moderate correlation.}$$

Eigenvalues

$$\det \begin{bmatrix} 2 - \lambda & 1 \\ 1 & 2 - \lambda \end{bmatrix} = 0$$

$$(2 - \lambda)^2 - 1 = 0$$

$$\lambda^2 - 4\lambda + 3 = 0$$

$$(\lambda - 3)(\lambda - 1) = 0$$

$$\boxed{\lambda_1 = 3, \quad \lambda_2 = 1}$$

Check: $3 + 1 = 4 = \text{tr}(C)$ ✓

Eigenvector for $\lambda_1 = 3$

$$\begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \mathbf{u} = \mathbf{0} \Rightarrow u_1 = u_2$$

$$\mathbf{u}_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Eigenvector for $\lambda_2 = 1$

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \mathbf{u} = \mathbf{0} \Rightarrow u_2 = -u_1$$

$$\mathbf{u}_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Orthogonality

$$\mathbf{u}_1 \cdot \mathbf{u}_2 = \frac{1}{2} - \frac{1}{2} = 0 \quad \checkmark$$

Project onto PC1

$$z_i = \frac{1}{\sqrt{2}}(\tilde{x}_{i1} + \tilde{x}_{i2})$$

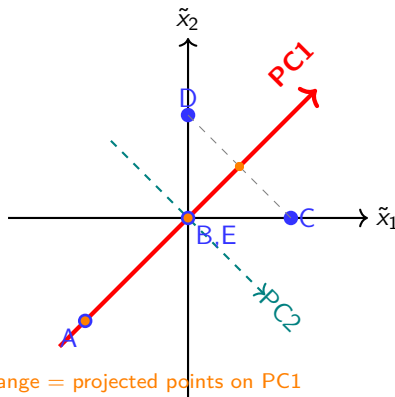
Pt	$\tilde{x}_1$	$\tilde{x}_2$	z
A	-2	-2	$-2\sqrt{2} \approx -2.83$
B	0	0	0
C	2	0	$\sqrt{2} \approx 1.41$
D	0	2	$\sqrt{2} \approx 1.41$
E	0	0	0

Explained Variance

$$\text{EVR}_1 = \frac{3}{4} = \mathbf{75\%}$$

$$\text{EVR}_2 = \frac{1}{4} = 25\%$$

Information loss = 25%



Note: C and D project to the **same point** on PC1 ($z = \sqrt{2}$) — this is where 25% information is lost.

Property	Example 1	Example 2
Data points	5 in $\mathbb{R}^2$	5 in $\mathbb{R}^2$
Correlation r	1.0 (perfect)	0.5 (moderate)
Covariance matrix	$\begin{bmatrix} 2.5 & 2.5 \\ 2.5 & 2.5 \end{bmatrix}$	$\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$
λ_1, λ_2	5, 0	3, 1
PC1 direction	$(1/\sqrt{2}, 1/\sqrt{2})$	$(1/\sqrt{2}, 1/\sqrt{2})$
Variance by PC1	100%	75%
Information loss	0%	25%

Key Takeaway

Higher correlation $\Rightarrow$ PCA compresses better. Perfect correlation $\Rightarrow$ lossless reduction by one dimension.

In Real Data

You rarely get 100% explained variance from a single PC. The goal is *acceptable* loss — typically retaining 90–95%.

Problem

Given the following dataset, perform PCA. Find the principal components, eigenvalues, and project the data onto PC1.

Point	x_1	x_2
P1	1	3
P2	3	5
P3	5	7
P4	2	4
P5	4	6

Tasks

- 1 Compute the mean of each feature and center the data.
- 2 Compute the 2×2 covariance matrix.
- 3 Find the eigenvalues and eigenvectors.
- 4 What percentage of variance does PC1 explain?
- 5 Project all points onto PC1 and write the 1D coordinates.

Problem

A PCA on a 4-feature dataset yields the following eigenvalues:

$$\lambda_1 = 8.2, \quad \lambda_2 = 3.5, \quad \lambda_3 = 1.1, \quad \lambda_4 = 0.2$$

Tasks

- 1 What is the total variance in the original data?
- 2 Compute the Explained Variance Ratio for each PC.
- 3 How many components should you retain for $\geq 90\%$ explained variance?
- 4 If you keep only PC1, what is the reconstruction error (in terms of variance)?
- 5 Draw a rough scree plot and mark the “elbow.”

Hint

Total variance = $\sum \lambda_i$. Reconstruction error when keeping k components = $\sum_{i=k+1}^p \lambda_i$.

Problem

The covariance matrix of a 3-feature centered dataset ($n = 100$) is:

$$C = \begin{bmatrix} 5 & 3 & 0 \\ 3 & 4 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Tasks

- 1 Find the eigenvalues of C .
Hint: One eigenvalue is obvious from the matrix structure. For the remaining 2×2 block, use the characteristic equation.
- 2 Find the corresponding eigenvectors.
- 3 What % of variance is retained by keeping 2 PCs?
- 4 What does the structure $\text{Cov}(x_3, x_1) = \text{Cov}(x_3, x_2) = 0$ tell you?

Hints

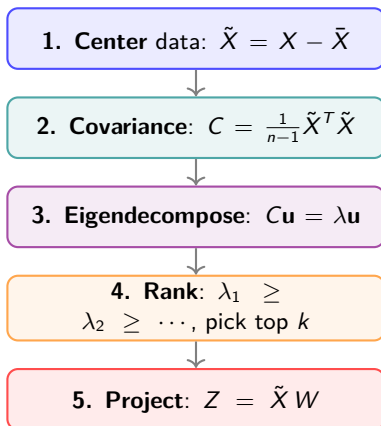
- The matrix is block-diagonal: $\begin{bmatrix} 5 & 3 \\ 3 & 4 \end{bmatrix}$ and $[1]$
- For the 2×2 block: $\lambda^2 - 9\lambda + 11 = 0$
- Feature 3 is **uncorrelated** with features 1 and 2 — it is already a natural axis

Answer the following (no computation required):

- 1 **Why must we center the data before PCA?** What happens if we skip centering?
- 2 **True or False:** PCA can discover non-linear relationships in data. Justify.
- 3 A dataset has 10 features, all with **zero covariance** with each other. How many principal components will PCA find, and what will the eigenvectors look like?
- 4 **When should you use the correlation matrix** instead of the covariance matrix for PCA? Give a concrete example.
- 5 A dataset has eigenvalues $\lambda_1 = 100$, $\lambda_2 = 0.01$, $\lambda_3 = 0.005$. A colleague says “We need all 3 PCs because every eigenvalue is nonzero.” Do you agree? Why or why not?
- 6 **Can PCA increase the number of dimensions?** That is, can you compute more PCs than original features? Explain.

Tip

These are common exam and interview questions. Focus on *why* PCA works, not just *how*.



Key Equations

- Objective: $\max \mathbf{u}^T \mathbf{C} \mathbf{u}, \|\mathbf{u}\| = 1$
- Solution: eigenvectors of $\mathbf{C}$
- Variance along PC_i : λ_i
- $\text{EVR}_i = \lambda_i / \sum \lambda_j$

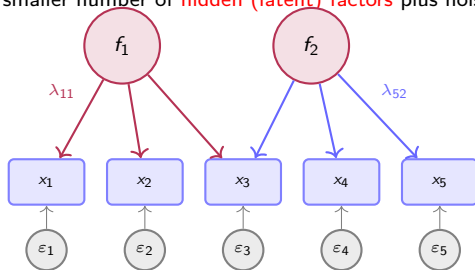
Remember

- PCA is **linear** and **unsupervised**
- **Standardize** if features have different scales
- PCs are orthogonal & uncorrelated
- Total variance is always preserved

- 1 Feature Extraction
 - Principal Component Analysis (PCA)
 - Linear Discriminant Analysis (LDA)
 - SVD, t-SNE, UMAP, and Autoencoders
- 2 Feature Construction
 - Mathematical and Polynomial Features
 - Temporal and Text Features
 - Aggregation, Binning, and Encoding
 - Interaction and Domain-Specific Features
- 3 Feature Selection
 - Filter Methods
 - Wrapper Methods
 - Embedded Methods
- 4 Principal Component Analysis (PCA): Introduction and The Mathematics
- 5 Factor Analysis**
- 6 Discriminant Analysis
- 7 Kernel PCA
- 8 t-SNE (t-distributed Stochastic Neighbor Embedding)
- 9 Advanced Feature Engineering for Multivariate Data

Factor Analysis (FA) assumes observed variables are generated by a smaller number of **hidden (latent) factors** plus noise.

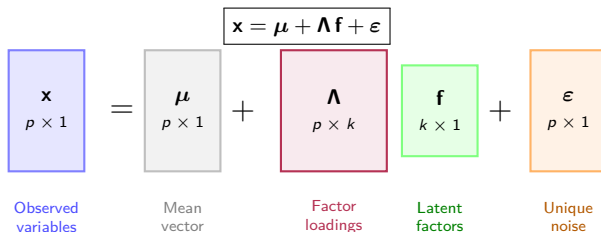
- Observed variables are **effects**, latent factors are the **causes**.
- FA is a **generative model**: it explains *how* data was produced.
- Decomposes variance into **shared** (communality) and **unique** (noise) parts.



PCA vs Factor Analysis

PCA asks: “What axes capture maximum variance?” **FA asks:** “What latent factors *generated* my data?”

The formal generative model for Factor Analysis:

$$\mathbf{x} = \boldsymbol{\mu} + \boldsymbol{\Lambda} \mathbf{f} + \boldsymbol{\varepsilon}$$


Observed variables Mean vector Factor loadings Latent factors Unique noise

Key Assumptions:

- $E[\mathbf{f}] = \mathbf{0}$, $\text{Cov}(\mathbf{f}) = \mathbf{I}$
- $E[\boldsymbol{\varepsilon}] = \mathbf{0}$, $\text{Cov}(\boldsymbol{\varepsilon}) = \boldsymbol{\Psi}$ (diagonal)
- $\text{Cov}(\mathbf{f}, \boldsymbol{\varepsilon}) = \mathbf{0}$
- Usually $k \ll p$ (few factors, many variables)

Implied Covariance Structure

$$\boldsymbol{\Sigma} = \boldsymbol{\Lambda} \boldsymbol{\Lambda}^T + \boldsymbol{\Psi} \iff \underbrace{\text{Total Covariance}}_{\boldsymbol{\Sigma}} = \underbrace{\text{Shared (Communality)}}_{\boldsymbol{\Lambda} \boldsymbol{\Lambda}^T} + \underbrace{\text{Unique (Noise)}}_{\boldsymbol{\Psi}}$$

PCA vs Factor Analysis — Detailed Comparison

Aspect	PCA	Factor Analysis
Goal	Maximize total variance	Discover latent structure
Model type	Deterministic transformation	Probabilistic generative model
Decomposes	Total variance	Shared vs. unique variance
Covariance	$\Sigma = \mathbf{V}\mathbf{\Lambda}\mathbf{V}^T$	$\Sigma = \mathbf{\Lambda}\mathbf{\Lambda}^T + \Psi$
Uniqueness	None — all variance explained	Each variable has unique ψ_i
Rotation	Components fixed (by variance)	Factors can be rotated
Interpretability	Ordered by variance	Designed for interpretation
Number of solutions	Unique	Infinite (rotation invariance)

PCA: Components = axes of max spread

FA: Factors = hidden causes of data

Loading matrix Λ tells us how strongly each factor influences each variable.

Communality of variable i :

$$h_i^2 = \sum_{j=1}^k \lambda_{ij}^2$$

Proportion of variance in x_i explained by the factors.

Uniqueness of variable i :

$$\psi_i = 1 - h_i^2$$

Proportion *not* explained (noise specific to that variable).

Variable	F_1	F_2	h^2
Math	0.89	0.12	0.81
Physics	0.85	0.20	0.76
Chemistry	0.78	0.25	0.67
English	0.15	0.88	0.80
History	0.22	0.82	0.72
Art	0.08	0.79	0.63

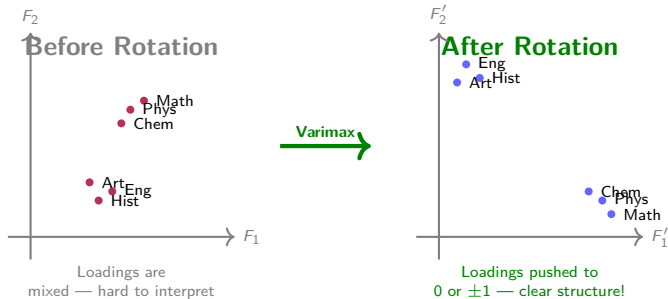
F_1 = Quantitative Ability

F_2 = Verbal/Creative Ability

Examples

Math: $h^2 = 0.89^2 + 0.12^2 = 0.792 + 0.014 = 0.81$. So 81% of Math's variance is explained by the two factors; 19% is unique noise.

The FA solution is **not unique**: if $\mathbf{\Lambda}$ is a solution, then $\mathbf{\Lambda}\mathbf{R}$ for any rotation $\mathbf{R}$ is also valid because $\mathbf{\Lambda}\mathbf{R}\mathbf{R}^T\mathbf{\Lambda}^T = \mathbf{\Lambda}\mathbf{\Lambda}^T$.



Orthogonal Rotations

Factors remain uncorrelated.

Varimax (most common): maximizes variance of squared loadings within each factor.

Oblique Rotations

Factors **can** correlate.

Promax, **Oblimin**: more realistic in social sciences where abilities are correlated.

Estimation Methods

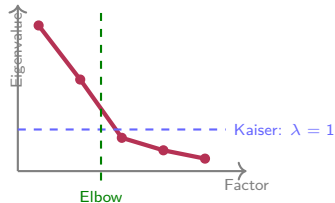
Maximum Likelihood (ML): Assumes normality of $\mathbf{f}$ and ϵ .
Maximizes:

$$\log L \propto -\frac{n}{2} \left[\log |\mathbf{\Lambda}\mathbf{\Lambda}^T + \mathbf{\Psi}| + \text{tr} \left((\mathbf{\Lambda}\mathbf{\Lambda}^T + \mathbf{\Psi})^{-1} \mathbf{S} \right) \right]$$

Allows χ^2 tests for number of factors.

Principal Axis Factoring (PAF): Iterative. Replace diagonal of $\mathbf{R}$ with communality estimates, extract, recalculate, repeat.

Choosing k (Number of Factors)



- **Kaiser:** eigenvalues > 1
- **Scree:** look for the “elbow”
- **Parallel analysis** (gold standard): compare with random data eigenvalues
- **ML χ^2 test:** formal hypothesis test

Given: Correlation matrix for 4 variables, extract 2 factors.

$$R = \begin{pmatrix} 1.00 & 0.80 & 0.30 & 0.25 \\ 0.80 & 1.00 & 0.25 & 0.20 \\ 0.30 & 0.25 & 1.00 & 0.70 \\ 0.25 & 0.20 & 0.70 & 1.00 \end{pmatrix}$$

Suppose after extraction, the loading matrix is:

	F_1	F_2	h^2	ψ_i
x_1	0.88	0.15	?	?
x_2	0.82	0.10	?	?
x_3	0.18	0.84	?	?
x_4	0.12	0.78	?	?

Calculate communality for x_1 :

$$h_1^2 = 0.88^2 + 0.15^2 = 0.7744 + 0.0225 = \mathbf{0.797}$$

Uniqueness for x_1 :

$$\psi_1 = 1 - 0.797 = \mathbf{0.203}$$

Verify reproduced correlation r_{12} :

$$\hat{r}_{12} = (0.88)(0.82) + (0.15)(0.10) = 0.722 + 0.015 = \mathbf{0.737}$$

(Actual $r_{12} = 0.80$; residual = $0.80 - 0.737 = 0.063$)

Interpretation : F_1 loads heavily on x_1, x_2 (a “group 1” factor). F_2 loads heavily on x_3, x_4 (a “group 2” factor). The residual (0.063) indicates reasonable but not perfect fit.

Complete the table and verify the implied covariance:

	F_1	F_2	h^2	ψ_i
x_1	0.88	0.15	0.797	0.203
x_2	0.82	0.10	0.682	0.318
x_3	0.18	0.84	0.738	0.262
x_4	0.12	0.78	0.622	0.378

Calculations for x_3 :

$$h_3^2 = 0.18^2 + 0.84^2 = 0.032 + 0.706 = 0.738$$

$$\psi_3 = 1 - 0.738 = 0.262$$

Reproduced correlations:

$$\hat{r}_{34} = (0.18)(0.12) + (0.84)(0.78)$$

$$= 0.022 + 0.655 = 0.677$$

Actual $r_{34} = 0.70$, residual = 0.023 ✓

$$\hat{r}_{13} = (0.88)(0.18) + (0.15)(0.84)$$

$$= 0.158 + 0.126 = 0.284$$

Actual $r_{13} = 0.30$, residual = 0.016 ✓

Goodness of Fit

Small residuals (< 0.05) across all pairs indicate the 2-factor model is a good fit. Total variance explained = $\frac{0.797+0.682+0.738+0.622}{4} = 70.97\%$.

Factor Analysis with scikit-learn

```
from sklearn.decomposition import FactorAnalysis
from sklearn.preprocessing import StandardScaler
import numpy as np
# Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Fit 2-factor model
fa = FactorAnalysis(n_components=2,
rotation='varimax', random_state=42)
fa.fit(X_scaled)
# Loading matrix (p x k)
loadings = fa.components_.T
print("Loadings:\n", loadings.round(3))
# Uniqueness (noise variance per variable)
print("Uniqueness:", fa.noise_variance_.round(3))
# Factor scores for each sample
scores = fa.transform(X_scaled)
```

Key Outputs

`components_`: loading matrix ($k \times p$, transpose for standard form).

`noise_variance_`: uniqueness ψ_i for each variable.

`transform()`: compute factor scores for each sample.

Rotation

Use `rotation='varimax'` for orthogonal rotation. For oblique, use the `factor_analyzer` library.

Use FA When

- You believe hidden latent variables generate your data (psychology, education, surveys).
- Interpreting the underlying structure matters more than pure compression.
- You need to name and label factors.
- Variables naturally cluster into groups.

Avoid FA When

- Goal is just dimensionality reduction (use PCA instead).
- Sample size is small ($n < 5p$ is risky).
- Data is non-continuous (binary, ordinal) without proper handling.
- No theoretical reason to expect latent factors.

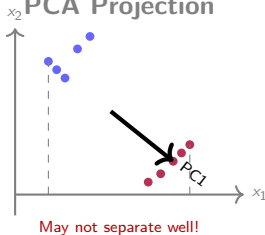
Classic Applications

Psychology: IQ test scores → verbal, quantitative, spatial factors. **Marketing:** Survey items → satisfaction, loyalty, quality dimensions. **Finance:** Stock returns → market, sector, momentum factors.

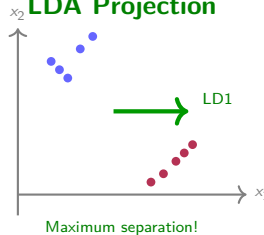
- 1 Feature Extraction
 - Principal Component Analysis (PCA)
 - Linear Discriminant Analysis (LDA)
 - SVD, t-SNE, UMAP, and Autoencoders
- 2 Feature Construction
 - Mathematical and Polynomial Features
 - Temporal and Text Features
 - Aggregation, Binning, and Encoding
 - Interaction and Domain-Specific Features
- 3 Feature Selection
 - Filter Methods
 - Wrapper Methods
 - Embedded Methods
- 4 Principal Component Analysis (PCA): Introduction and The Mathematics
- 5 Factor Analysis
- 6 Discriminant Analysis**
- 7 Kernel PCA
- 8 t-SNE (t-distributed Stochastic Neighbor Embedding)
- 9 Advanced Feature Engineering for Multivariate Data

Discriminant Analysis is a **supervised** technique that finds projections maximizing **class separation**.

PCA Projection



LDA Projection



$$J(\mathbf{w}) = \frac{\mathbf{w}^T \mathbf{S}_B \mathbf{w}}{\mathbf{w}^T \mathbf{S}_W \mathbf{w}} \rightarrow \text{Maximize } \frac{\text{Between-class scatter}}{\text{Within-class scatter}}$$

Key Difference from PCA

PCA is **unsupervised** (ignores labels). LDA is **supervised** (uses class labels to find the most discriminative directions).

LDA relies on two key matrices that measure spread *within* and *between* classes.

Within-Class Scatter S_W :

$$S_W = \sum_{c=1}^g \sum_{\mathbf{x}_i \in C_c} (\mathbf{x}_i - \boldsymbol{\mu}_c)(\mathbf{x}_i - \boldsymbol{\mu}_c)^T$$

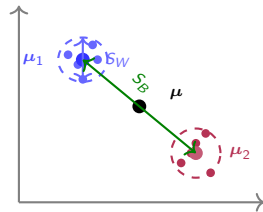
Measures how spread out each class is internally. We want this to be **small**.

Between-Class Scatter S_B :

$$S_B = \sum_{c=1}^g n_c (\boldsymbol{\mu}_c - \boldsymbol{\mu})(\boldsymbol{\mu}_c - \boldsymbol{\mu})^T$$

Measures how far apart the class means are. We want this to be **large**.

Generalized Eigenvalue Problem : $\mathbf{S}_W^{-1} \mathbf{S}_B \mathbf{w} = \lambda \mathbf{w}$. The eigenvectors of $\mathbf{S}_W^{-1} \mathbf{S}_B$ give the discriminant directions. At most $\min(p, g - 1)$ non-zero eigenvalues.



Good LDA:
Small S_W , Large S_B

Given data for two classes:

Class 1:

x_1	x_2
4	2
2	4
2	3
3	6
4	4

 $\mu_1 = \begin{pmatrix} 3 \\ 3.8 \end{pmatrix}$

Class 2:

x_1	x_2
9	10
6	8
9	5
8	7
10	8

 $\mu_2 = \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix}$

Step 1: Compute class means.

$$\mu_1 = \begin{pmatrix} 3.0 \\ 3.8 \end{pmatrix}, \quad \mu_2 = \begin{pmatrix} 8.4 \\ 7.6 \end{pmatrix}$$

Mean difference:

$$\mu_2 - \mu_1 = \begin{pmatrix} 5.4 \\ 3.8 \end{pmatrix}$$

Step 2: Within-class scatter matrices (sum of deviations from class mean):

$$\mathbf{S}_1 = \begin{pmatrix} 4.0 & -1.0 \\ -1.0 & 8.8 \end{pmatrix}, \quad \mathbf{S}_2 = \begin{pmatrix} 10.8 & 0.2 \\ 0.2 & 14.8 \end{pmatrix}, \quad \mathbf{S}_W = \mathbf{S}_1 + \mathbf{S}_2 = \begin{pmatrix} 14.8 & -0.8 \\ -0.8 & 23.6 \end{pmatrix}$$

Next: Compute $\mathbf{S}_W^{-1}$ and the LDA direction $\mathbf{w}$

For binary LDA, $\mathbf{w} = \mathbf{S}_W^{-1}(\mu_1 - \mu_2)$ — no eigenvalue decomposition needed!

Step 3: Compute $\mathbf{S}_W^{-1}$:

$$\mathbf{S}_W = \begin{pmatrix} 14.8 & -0.8 \\ -0.8 & 23.6 \end{pmatrix}, \quad \det(\mathbf{S}_W) = (14.8)(23.6) - (-0.8)^2 = 349.28 - 0.64 = 348.64$$

$$\mathbf{S}_W^{-1} = \frac{1}{348.64} \begin{pmatrix} 23.6 & 0.8 \\ 0.8 & 14.8 \end{pmatrix} = \begin{pmatrix} 0.0677 & 0.0023 \\ 0.0023 & 0.0425 \end{pmatrix}$$

Step 4: LDA direction (for 2 classes):

$$\mathbf{w} = \mathbf{S}_W^{-1}(\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2) = \begin{pmatrix} 0.0677 & 0.0023 \\ 0.0023 & 0.0425 \end{pmatrix} \begin{pmatrix} -5.4 \\ -3.8 \end{pmatrix} = \begin{pmatrix} -0.374 \\ -0.174 \end{pmatrix}$$

Step 5: Project data onto $\mathbf{w}$. For sample (4, 2) from Class 1:

$$z = \mathbf{w}^T \mathbf{x} = (-0.374)(4) + (-0.174)(2) = -1.496 - 0.348 = -1.844$$

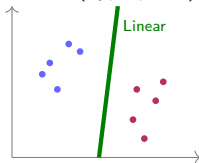
Result

Class 1 projections cluster around $z \approx -1.78$; Class 2 around $z \approx -4.78$. The classes are well-separated on this single discriminant axis.

LDA: Shared Covariance

Assumes $\Sigma_1 = \Sigma_2 = \dots = \Sigma$

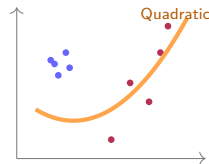
Decision boundary is **linear** (hyperplane).



QDA: Different Covariances

Each class has its own Σ_c

Decision boundary is **quadratic** (curved).



QDA Decision Rule for class k :

$$\delta_k(\mathbf{x}) = -\frac{1}{2} \log |\Sigma_k| - \frac{1}{2} (\mathbf{x} - \mu_k)^T \Sigma_k^{-1} (\mathbf{x} - \mu_k) + \log \pi_k$$

Trade-off

QDA is more flexible but needs more data to estimate each class covariance. **Regularized DA** interpolates:

$$\Sigma_k(\alpha) = \alpha \Sigma_k + (1 - \alpha) \Sigma_{\text{pooled}}.$$

Assumptions of LDA:

- Multivariate **normality** within each class.
- **Equal covariance** matrices across classes ($\Sigma_1 = \Sigma_2 = \dots$).
- S_W must be **invertible** (requires $n > p$).
- Features are **continuous** (not categorical).

When $p > n$: S_W is singular!

- PCA first, then LDA.
- Regularized LDA (shrink toward identity).
- Penalized LDA.

Comparison Summary:

Aspect	LDA	QDA
Boundary	Linear	Quadratic
Cov. assumption	Equal	Different
Parameters	Fewer	More
Flexibility	Low	High
Data needed	Less	More
Overfitting risk	Lower	Higher

Max LDA components
 $= \min(p, g - 1)$
 3 classes $\Rightarrow$ at most 2 LDA dims

Practical Rule

Use **LDA** when classes have similar spread and you have limited data. Use **QDA** when class shapes visibly differ and you have abundant samples.

LDA & QDA with scikit-learn

```
from sklearn.discriminant_analysis import (
    LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis)

# LDA (classification + reduction)
lda = LinearDiscriminantAnalysis(
    n_components=2)
X_lda = lda.fit_transform(X_train, y_train)
y_pred = lda.predict(X_test)

# QDA (classification only)
qda = QuadraticDiscriminantAnalysis()
qda.fit(X_train, y_train)
y_pred_qda = qda.predict(X_test)

# Regularized LDA (for near-singular  $S_W$ )
lda_reg = LinearDiscriminantAnalysis(
    solver='lsqr', shrinkage='auto')
lda_reg.fit(X_train, y_train)
```

LDA: Dual Purpose

LDA can be used for both **dimensionality reduction** (transform) and **classification** (predict). PCA can only do reduction.

Shrinkage

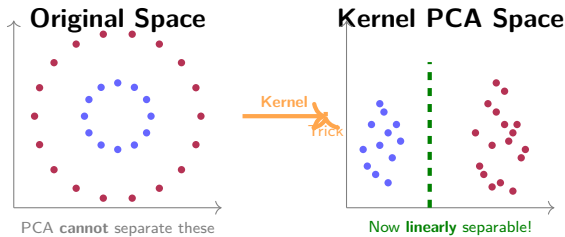
`shrinkage='auto'` uses the Ledoit-Wolf lemma to automatically regularize S_W when p is large relative to n .

Examples

For a 5-class problem with 50 features: LDA reduces to at most 4 dimensions — massive compression with class-awareness!

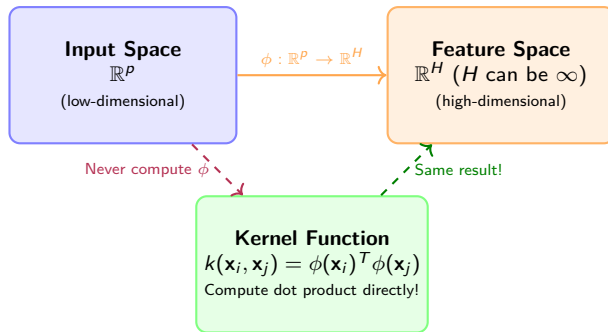
- 1 Feature Extraction
 - Principal Component Analysis (PCA)
 - Linear Discriminant Analysis (LDA)
 - SVD, t-SNE, UMAP, and Autoencoders
- 2 Feature Construction
 - Mathematical and Polynomial Features
 - Temporal and Text Features
 - Aggregation, Binning, and Encoding
 - Interaction and Domain-Specific Features
- 3 Feature Selection
 - Filter Methods
 - Wrapper Methods
 - Embedded Methods
- 4 Principal Component Analysis (PCA): Introduction and The Mathematics
- 5 Factor Analysis
- 6 Discriminant Analysis
- 7 Kernel PCA**
- 8 t-SNE (t-distributed Stochastic Neighbor Embedding)
- 9 Advanced Feature Engineering for Multivariate Data

Standard PCA finds **linear** subspaces. But what if data lies on a **non-linear manifold**?



Core Strategy

Step 1: Map data to high-dimensional space via $\phi(\mathbf{x})$. **Step 2:** Perform linear PCA there. **Step 3:** Use the **kernel trick** to avoid computing ϕ explicitly.



In standard PCA, we eigendecompose the covariance. In Kernel PCA, we eigendecompose the $n \times n$ **kernel matrix**:

$$\mathbf{K}\boldsymbol{\alpha} = n\lambda\boldsymbol{\alpha} \quad \text{where } K_{ij} = k(\mathbf{x}_i, \mathbf{x}_j)$$

Projection of a new point $\mathbf{x}$ onto the m -th component: $\text{PC}_m(\mathbf{x}) = \sum_{i=1}^n \alpha_i^{(m)} k(\mathbf{x}_i, \mathbf{x})$ **Key Insight** We only ever compute dot products $k(\mathbf{x}_i, \mathbf{x}_j)$ — never the mapping ϕ itself. This is the “trick.”

Kernel	Formula	Feature Space	Character
Linear	$k(\mathbf{x}, \mathbf{y}) = \mathbf{x}^T \mathbf{y}$	Same as input	Standard PCA
Polynomial	$k(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y} + c)^d$	Finite, degree d	Polynomial interactions
RBF (Gaussian)	$k(\mathbf{x}, \mathbf{y}) = e^{-\gamma \ \mathbf{x} - \mathbf{y}\ ^2}$	Infinite!	Most flexible
Sigmoid	$k(\mathbf{x}, \mathbf{y}) = \tanh(\alpha \mathbf{x}^T \mathbf{y} + c)$	—	Neural-inspired

RBF Parameter γ

Small γ : smooth, global influence (each point affects many others).

Large γ : local influence (very tight neighborhoods).

Centering in Feature Space

Must center the kernel matrix:

$$\tilde{\mathbf{K}} = \mathbf{K} - \mathbf{1}_n \mathbf{K} - \mathbf{K} \mathbf{1}_n + \mathbf{1}_n \mathbf{K} \mathbf{1}_n$$

where $\mathbf{1}_n$ is the $n \times n$ matrix with all entries $\frac{1}{n}$.

Given: 4 data points in $\mathbb{R}^2$: $\mathbf{x}_1 = (1, 0)$, $\mathbf{x}_2 = (0, 1)$, $\mathbf{x}_3 = (-1, 0)$, $\mathbf{x}_4 = (0, -1)$.

Using **polynomial kernel** $k(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^T \mathbf{y})^2$ (degree 2, $c = 0$):

Step 1: Compute kernel matrix K

$$k(\mathbf{x}_1, \mathbf{x}_1) = (1 \cdot 1 + 0 \cdot 0)^2 = 1$$

$$k(\mathbf{x}_1, \mathbf{x}_2) = (1 \cdot 0 + 0 \cdot 1)^2 = 0$$

$$k(\mathbf{x}_1, \mathbf{x}_3) = (1 \cdot (-1) + 0 \cdot 0)^2 = 1$$

$$k(\mathbf{x}_1, \mathbf{x}_4) = (1 \cdot 0 + 0 \cdot (-1))^2 = 0$$

$$\mathbf{K} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}$$

Step 2: Eigendecompose K

Eigenvalues: $\lambda_1 = 2$, $\lambda_2 = 2$, $\lambda_3 = 0$, $\lambda_4 = 0$

Eigenvectors (for $\lambda = 2$):

$$\boldsymbol{\alpha}^{(1)} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 0 \\ 1 \\ 0 \end{pmatrix}, \quad \boldsymbol{\alpha}^{(2)} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \\ 1 \\ 0 \\ 1 \end{pmatrix}$$

Step 3: Normalize: $\tilde{\boldsymbol{\alpha}}^{(m)} = \frac{\boldsymbol{\alpha}^{(m)}}{\sqrt{n \lambda_m}} = \frac{\boldsymbol{\alpha}^{(m)}}{\sqrt{4 \cdot 2}} = \frac{\boldsymbol{\alpha}^{(m)}}{2\sqrt{2}}$

Projection of $\mathbf{x}_1 = (1, 0)$ onto first kernel PC

$$\text{PC}_1(\mathbf{x}_1) = \sum_i \tilde{\alpha}_i^{(1)} k(\mathbf{x}_i, \mathbf{x}_1) = \frac{1}{2\sqrt{2}} (1 \cdot 1 + 0 \cdot 0 + 1 \cdot 1 + 0 \cdot 0) = \frac{2}{2\sqrt{2}} = \frac{1}{\sqrt{2}} \approx 0.707$$

Aspect	Standard PCA	Kernel PCA
Relationships	Linear only	Non-linear
Eigendecompose	Covariance $\mathbf{C}$ ($p \times p$)	Kernel $\mathbf{K}$ ($n \times n$)
Reconstruction	$\hat{\mathbf{x}} = \mathbf{W}\mathbf{W}^T \mathbf{x}$ (easy)	Requires iterative pre-image
Explained variance	Clear (eigenvalue ratios)	Not directly interpretable
Scalability	$O(p^3)$ or $O(np^2)$	$O(n^3)$ — expensive for large n
Hyperparameters	None	Kernel type + kernel parameters

When to Use Kernel PCA

Data has clear non-linear structure (swiss roll, concentric circles, spirals). Standard PCA fails visually.

Limitation

Choosing the right kernel and tuning γ requires experimentation. No explained variance ratio to guide component selection.

Kernel PCA with scikit-learn

```
from sklearn.decomposition import KernelPCA
from sklearn.datasets import make_circles
import matplotlib.pyplot as plt
# Non-linear data: concentric circles
X, y = make_circles(n_samples=400, factor=0.3, noise
                    =0.05)
# Standard PCA (fails)
from sklearn.decomposition import PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
# Kernel PCA with RBF kernel
kpca = KernelPCA(n_components=2, kernel='rbf', gamma=10)
X_kpca = kpca.fit_transform(X)
# Polynomial kernel
kpca_poly = KernelPCA(n_components=2,
                      kernel='poly', degree=3, coef0=1)
X_kpoly = kpca_poly.fit_transform(X)
```

Key Parameters

kernel: 'rbf', 'poly', 'sigmoid', 'linear', or custom callable.

gamma: RBF width ($\frac{1}{2\sigma^2}$). Start with $1/n_{\text{features}}$.

degree, coef0: for polynomial kernel.

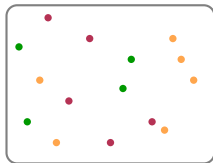
Tip

Try multiple γ values. Too small $\rightarrow$ all points look similar. Too large $\rightarrow$ only nearest neighbors matter.

- 1 Feature Extraction
 - Principal Component Analysis (PCA)
 - Linear Discriminant Analysis (LDA)
 - SVD, t-SNE, UMAP, and Autoencoders
- 2 Feature Construction
 - Mathematical and Polynomial Features
 - Temporal and Text Features
 - Aggregation, Binning, and Encoding
 - Interaction and Domain-Specific Features
- 3 Feature Selection
 - Filter Methods
 - Wrapper Methods
 - Embedded Methods
- 4 Principal Component Analysis (PCA): Introduction and The Mathematics
- 5 Factor Analysis
- 6 Discriminant Analysis
- 7 Kernel PCA
- 8 t-SNE (t-distributed Stochastic Neighbor Embedding)
- 9 Advanced Feature Engineering for Multivariate Data

t-SNE maps high-dimensional data to 2D/3D by preserving **local neighborhood structure** with remarkable visual clarity.

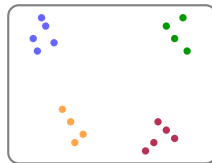
High-D (e.g., 784 pixels)



Structure hidden

t-SNE →

2D Embedding



Clusters revealed!

What Makes t-SNE Special?

Unlike PCA (linear, global variance), t-SNE focuses on **preserving which points are neighbors** in high-D, making it ideal for discovering clusters in complex datasets like MNIST digits.

Step 1: Compute pairwise similarities p_{ij} in high-D (Gaussian)

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}$$

Step 2: Initialize random positions y_i in 2D

Step 3: Compute pairwise similarities q_{ij} in 2D (**t-distribution**)

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|y_i - y_k\|^2)^{-1}}$$

Step 4: Minimize $KL(P||Q)$ via gradient descent

$$KL = \sum_{ij} p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

Step 5: Repeat Steps 3–4 for 1000+ iterations

The Key Innovation: Student's t-Distribution in Low-D

Using a **heavy-tailed t-distribution** (instead of Gaussian) in the low-D space solves the “crowding problem” and lets clusters spread out naturally.

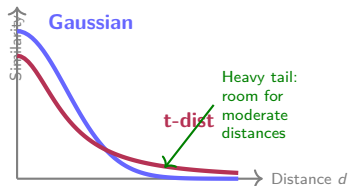
The Crowding Problem and Why the t-Distribution

The Problem:

In high-D, a point can have many equidistant neighbors. When mapping to 2D, there simply isn't enough room — moderate-distance points get **crushed** onto nearby points.

The Solution:

The t-distribution has **heavier tails** than Gaussian. Moderate distances in high-D can be represented as *larger* distances in 2D without paying a huge penalty.



$$\underbrace{e^{-d^2}}_{\text{Gaussian: drops fast}} \quad \text{vs.} \quad \underbrace{(1 + d^2)^{-1}}_{\text{t-dist: drops slowly (heavy tail)}}$$

Physical Analogy

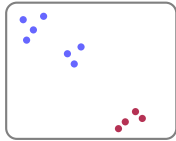
Think of each pair as connected by a spring. p_{ij} exerts **attraction**, q_{ij} exerts **repulsion**. Gradient descent finds equilibrium where neighbors stay close but non-neighbors are pushed apart.

Perplexity controls the effective number of neighbors each point considers. It is t-SNE's most important hyperparameter.

$$\text{Perp}(P_i) = 2^{H(P_i)} \quad \text{where} \quad H(P_i) = - \sum_j p_{j|i} \log_2 p_{j|i}$$

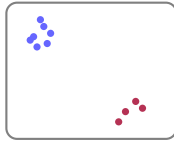
The σ_i for each point is found via binary search such that the perplexity matches the user-specified value.

Perplexity = 5



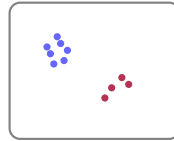
Very local — may split clusters

Perplexity = 30



Good balance — typical choice

Perplexity = 100



Too global — clusters may merge

Guidelines

Typical range: **5–50**. Use 30 as default. **Always try multiple perplexity values** and compare results. Perplexity should be less than n .

t-SNE — Critical Caveats (What It Does NOT Tell You)

× Cluster **sizes** are meaningless

t-SNE expands dense clusters and compresses sparse ones

× Distances **between** clusters are meaningless

Two far-apart clusters are not necessarily more different

× Number of clusters can be **misleading**

Wrong perplexity can split one cluster or merge two

× Results are **non-deterministic**

Different random seeds give different layouts

× Cannot embed **new data** (no transform)

Must rerun on entire dataset (use openTSNE for transform)

Golden Rule

Use t-SNE **only for visualization**. Never use t-SNE distances as features for a downstream model or to make quantitative claims about cluster relationships.

Given: 3 points in $\mathbb{R}^3$: $\mathbf{x}_1 = (0, 0, 0)$, $\mathbf{x}_2 = (1, 0, 0)$, $\mathbf{x}_3 = (5, 5, 5)$. Map to 1D.

Step 1: Pairwise distances

$$d_{12} = \|\mathbf{x}_1 - \mathbf{x}_2\| = 1$$

$$d_{13} = \|\mathbf{x}_1 - \mathbf{x}_3\| = \sqrt{75} \approx 8.66$$

$$d_{23} = \|\mathbf{x}_2 - \mathbf{x}_3\| = \sqrt{66} \approx 8.12$$

Step 2: High-D similarities (let $\sigma = 1$):

$$p_{2|1} = \frac{e^{-d_{12}^2/2}}{e^{-d_{12}^2/2} + e^{-d_{13}^2/2}} = \frac{e^{-0.5}}{e^{-0.5} + e^{-37.5}}$$

$$\approx \frac{0.607}{0.607+0} \approx 1.000$$

$$p_{3|1} \approx 0.000$$

Interpretation:

Point 1 “sees” Point 2 as its neighbor ($p \approx 1$) and Point 3 as irrelevant ($p \approx 0$).

Step 3: In the 2D/1D embedding, the algorithm will place y_1 and y_2 **close together** and y_3 **far away**, because the KL divergence penalizes:

- High p_{ij} with low q_{ij} (neighbors pulled apart — **strong penalty**)
- Low p_{ij} with high q_{ij} (non-neighbors pushed together — weak penalty)

Key Takeaway

This asymmetric penalty is why t-SNE excels at preserving **local** structure: it strongly penalizes tearing apart true neighbors.

Aspect	PCA	t-SNE	UMAP
Type	Linear	Non-linear	Non-linear
Preserves	Global variance	Local neighborhoods	Local + some global
Speed	Very fast	Slow $O(n^2)$	Fast
Deterministic	Yes	No	No
New data	Yes (transform)	No (refit)	Yes (transform)
Scalability	Millions	Thousands	Millions
Best for	Exploration	Visualization	Visualization + ML
Distances meaningful	Yes	No	Partially

Practical Advice

Start with **PCA** (fast, gives global picture). Use **t-SNE** for publication-quality cluster visualizations. Use **UMAP** when you need speed, new-point embedding, or better global structure.

t-SNE with scikit-learn

```
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
# Basic t-SNE
tsne = TSNE( n_components=2, perplexity=30,
learning_rate='auto', init='pca',      # stable
            initialization
n_iter=1000, random_state=42)
X_tsne = tsne.fit_transform(X_scaled)
# Visualize
plt.figure(figsize=(8,6))
scatter = plt.scatter(
X_tsne[:,0], X_tsne[:,1],
c=y, cmap='tab10', s=5, alpha=0.7)
plt.colorbar(scatter)
plt.title('t-SNE Visualization')
plt.show()
# Check convergence
print(f"KL divergence: "
f"{tsne.kl_divergence_:.4f}")
```

Best Practices

- Use `init='pca'` for stable results.
- Always run ≥ 1000 iterations.
- Try multiple perplexity values.
- PCA to 50 dims first if $p > 50$.

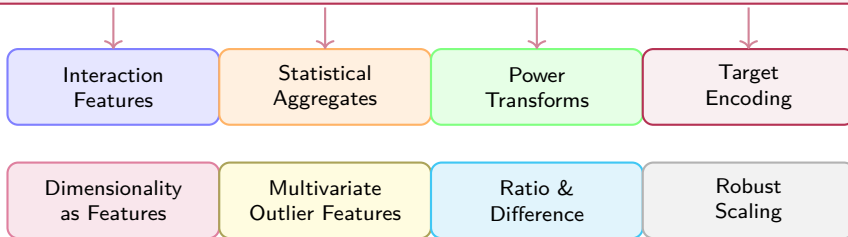
Multiple Perplexities

Always compare results across perplexity values (e.g., 5, 15, 30, 50). Only trust patterns that appear **consistently** across settings.

- 1 Feature Extraction
 - Principal Component Analysis (PCA)
 - Linear Discriminant Analysis (LDA)
 - SVD, t-SNE, UMAP, and Autoencoders
- 2 Feature Construction
 - Mathematical and Polynomial Features
 - Temporal and Text Features
 - Aggregation, Binning, and Encoding
 - Interaction and Domain-Specific Features
- 3 Feature Selection
 - Filter Methods
 - Wrapper Methods
 - Embedded Methods
- 4 Principal Component Analysis (PCA): Introduction and The Mathematics
- 5 Factor Analysis
- 6 Discriminant Analysis
- 7 Kernel PCA
- 8 t-SNE (t-distributed Stochastic Neighbor Embedding)
- 9 Advanced Feature Engineering for Multivariate Data

Beyond basic feature construction, multivariate data offers rich opportunities through **inter-feature relationships**.

ADVANCED FEATURE ENGINEERING FOR MULTIVARIATE DATA



Why “Advanced”?

These techniques go beyond single-feature transforms. They exploit the **multivariate structure**: correlations, distributions, and interactions between multiple features simultaneously.

Individual features may be weakly predictive, but their **combinations** can be powerful.

Types of Interaction Features:

- **Products:** $x_i \times x_j$ — multiplicative interactions.
- **Ratios:** x_i/x_j — normalized comparisons.
- **Differences:** $x_i - x_j$ — contrasts.
- **Min/Max:** $\min(x_i, x_j)$, $\max(x_i, x_j)$ — boundary effects.

Edu	Exp	Salary
PhD	10yr	High
PhD	2yr	Medium
HS	10yr	Medium
HS	2yr	Low

Feature explosion warning:

p features $\rightarrow \binom{p}{2} = \frac{p(p-1)}{2}$ pairs.

$p = 50 \rightarrow 1225$ interaction features!



Neither alone predicts salary well, but together they do!

Practical Strategy

Don't blindly create all pairs. Use domain knowledge, or first select important features, then create interactions among those.

When features form natural groups (e.g., multiple sensor readings, survey items), compute **row-wise statistics** across those features.

s_1	s_2	s_3	s_4	→	mean	std	range	skew	kurt
2.3	4.1	3.7	2.9	→	3.25	0.79	1.8	0.31	-1.2

Original features

Engineered aggregate features

Common aggregates across feature groups:

- Mean, Median
- Standard deviation
- Min, Max, Range
- Skewness, Kurtosis
- Count above/below threshold
- Percentiles (25th, 75th)

When This Helps

Multiple sensor channels, financial account features, multiple test scores, gene expression profiles — any time features represent **repeated measurements** of similar quantities.

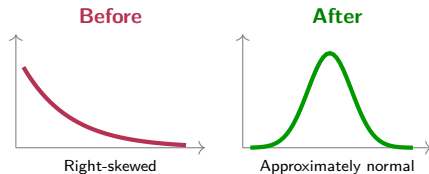
Power transforms find the **optimal transformation** to make features more Gaussian-like.

Box-Cox Transform (requires $x > 0$):

$$x^{(\lambda)} = \begin{cases} \frac{x^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \ln(x) & \text{if } \lambda = 0 \end{cases}$$

Yeo-Johnson Transform (any x):

$$x^{(\lambda)} = \begin{cases} \frac{(x+1)^\lambda - 1}{\lambda} & x \geq 0, \lambda \neq 0 \\ \ln(x+1) & x \geq 0, \lambda = 0 \\ -\frac{(-x+1)^{2-\lambda} - 1}{2-\lambda} & x < 0, \lambda \neq 2 \\ -\ln(-x+1) & x < 0, \lambda = 2 \end{cases}$$



λ is learned

The optimal λ is found via maximum likelihood. Different λ values recover common transforms: $\lambda = 0$ (log), $\lambda = 0.5$ (sqrt), $\lambda = -1$ (reciprocal).

Replace a categorical feature with the **mean of the target** for that category. Powerful for high-cardinality features but risks **data leakage**.

City	Target	City_enc
NYC	1	0.67
NYC	1	
NYC	0	
LA	0	0.50
LA	1	
SF	1	1.00

Problem: SF has only 1 sample! Encoding = 1.00 is overfit.

Smoothed Target Encoding:

$$\text{enc}(c) = \frac{n_c \cdot \bar{y}_c + m \cdot \bar{y}_{\text{global}}}{n_c + m}$$

where m is the smoothing parameter, n_c is category count, $\bar{y}_c$ is category mean, $\bar{y}_{\text{global}}$ is global mean.

Example (global mean $\bar{y} = 0.67$, $m = 3$):

$$\text{SF: } \frac{1 \cdot 1.0 + 3 \cdot 0.67}{1 + 3} = \frac{3.01}{4} = 0.75$$

$$\text{NYC: } \frac{3 \cdot 0.67 + 3 \cdot 0.67}{3 + 3} = 0.67$$

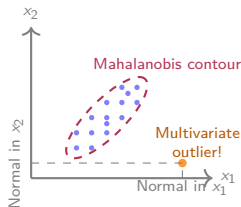
Rare categories are pulled toward the global mean.

Critical: Prevent Leakage

Always compute target encodings using **cross-validation folds**: for each fold, compute means on the *other* folds and apply to the held-out fold.

A point may not be an outlier in *any* single feature but can be a **multivariate outlier**.

$$D_M(\mathbf{x}) = \sqrt{(\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})}$$



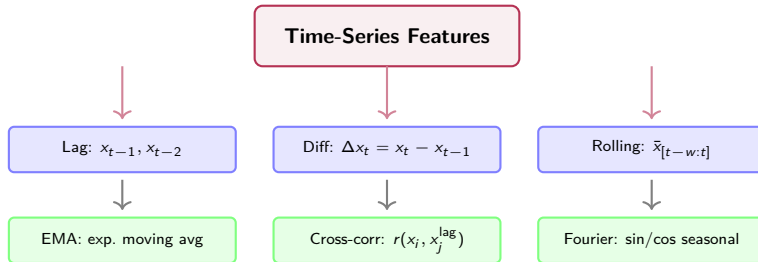
Use as Feature

Add $D_M(\mathbf{x})$ as a new feature. Samples with high Mahalanobis distance are unusual given the correlation structure.

Outlier Threshold

Under multivariate normality, $D_M^2 \sim \chi_p^2$. Flag points where $D_M^2 > \chi_{p,0.975}^2$.

When multivariate data has a temporal dimension, engineer features that capture **dynamics across time and variables**.



Cross-Variable Features

- Lag of variable j predicting variable i
- Rolling correlation between variables
- Ratio of rolling means across variables

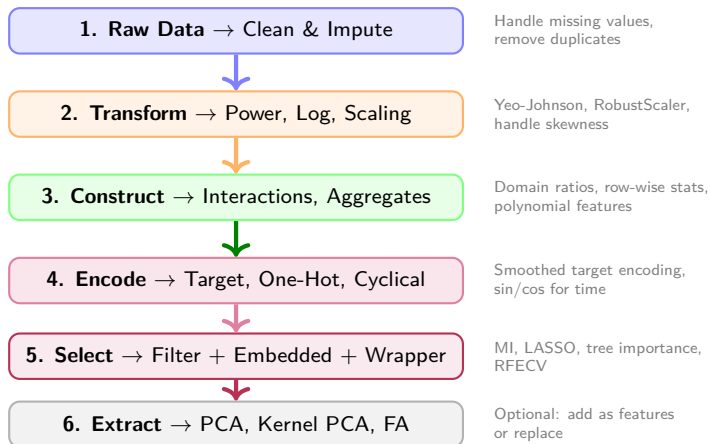
Fourier Features for Seasonality

$$x_{\sin,k} = \sin\left(\frac{2\pi k \cdot t}{T}\right)$$

$$x_{\cos,k} = \cos\left(\frac{2\pi k \cdot t}{T}\right)$$

where T is the period (24 for hourly, 7 for weekly, 365 for yearly).

Complete Feature Engineering Pipeline



Golden Rule: All transformations and encodings must be **fit on training data only** and then **applied** (not refit) to test/validation data to prevent data leakage.

Problem: Predict house prices from raw features: area, rooms, year_built, zip_code, sale_date.

Step	Technique	New Feature	Rationale
Transform	Log	<code>log_area = log1p(area)</code>	Right-skewed
Ratio	Division	<code>price_per_sqft = price/area</code>	Normalize by size
Interaction	Product	<code>area_x_rooms = area × rooms</code>	Joint effect
Temporal	Difference	<code>house_age = 2026 - year_built</code>	Age matters more
Cyclical	Sin/Cos	<code>month_sin, month_cos</code>	Seasonal effects
Encoding	Target	<code>zip_target_enc</code>	High cardinality
Aggregate	GroupBy	<code>zip_avg_price (from train)</code>	Neighborhood quality
Outlier	Mahalanobis	<code>mahal_dist</code>	Flag unusual combos

Result

Starting from 5 raw features, we now have 13+ engineered features. After LASSO selection, the top 8 features improve R^2 from **0.72** (raw only) to **0.91**.

Method	Type	Super-vised?	Preserves	Best For	Scalability
PCA	Linear	No	Global variance	General reduction	Excellent
Factor Analysis	Linear	No	Latent structure	Interpret. factors	Good
LDA	Linear	Yes	Class separation	Classification	Good
QDA	Quadratic	Yes	Class shapes	Curved boundaries	Moderate
Kernel PCA	Non-linear	No	Non-linear var.	Manifold data	Moderate
t-SNE	Non-linear	No	Local neighbors	Visualization only	Poor
Feature Eng.	Manual	Both	Domain knowl- edge	All tasks	Depends

Key Connections: PCA $\xrightarrow{\text{add labels}}$ LDA $\xrightarrow{\text{add kernels}}$ Kernel PCA $\xrightarrow{\text{add latent model}}$ FA $\xrightarrow{\text{add visualization}}$ t-SNE

- ❶ **Factor Analysis vs PCA:** Explain why the FA model $\mathbf{x} = \mathbf{\Lambda}\mathbf{f} + \boldsymbol{\epsilon}$ produces a different decomposition of the covariance matrix than PCA. What role does the uniqueness matrix $\boldsymbol{\Psi}$ play?
- ❷ **LDA Limitation:** Why can LDA produce at most $g - 1$ discriminant components for a g -class problem? What is the rank of $\mathbf{S}_B$?
- ❸ **Kernel PCA:** Explain why we eigendecompose the $n \times n$ kernel matrix $\mathbf{K}$ rather than the $p \times p$ covariance matrix. In what situation is this more efficient than standard PCA?
- ❹ **t-SNE Caveats:** A colleague shows you a t-SNE plot and claims “Cluster A is twice as large as Cluster B, and they are very far apart, so they represent very different populations.” Identify two errors in this interpretation.
- ❺ **Crowding Problem:** Explain the crowding problem in dimensionality reduction and how the t-distribution in t-SNE addresses it compared to using a Gaussian in the low-dimensional space.
- ❻ **Feature Engineering:** For a dataset with features `income`, `debt`, and `num_dependents`, propose 4 meaningful engineered features and justify each.

1 Factor Analysis Calculation:

Given the loading matrix below, compute the communality h_i^2 and uniqueness ψ_i for each variable. Also compute the reproduced correlation $\hat{r}_{12}$.

	F_1	F_2
x_1	0.90	0.20
x_2	0.75	0.35
x_3	0.10	0.85

2 LDA Projection:

Given two classes with means $\mu_1 = (2, 3)^T$, $\mu_2 = (6, 5)^T$, and pooled within-class scatter $\mathbf{S}_W = \begin{pmatrix} 4 & 1 \\ 1 & 2 \end{pmatrix}$, compute: (a) the LDA direction $\mathbf{w} = \mathbf{S}_W^{-1}(\mu_1 - \mu_2)$, and (b) the projection of a new point $\mathbf{x}_{\text{new}} = (4, 4)^T$.

3 Kernel Matrix:

For points $\mathbf{x}_1 = (1, 2)$, $\mathbf{x}_2 = (3, 4)$, $\mathbf{x}_3 = (0, 1)$, compute the full 3×3 kernel matrix using the RBF kernel $k(\mathbf{x}, \mathbf{y}) = e^{-0.5\|\mathbf{x}-\mathbf{y}\|^2}$.

4 t-SNE Hyperparameter Experiment:

Using the Iris dataset, run t-SNE with perplexity values of 5, 15, 30, and 50. For each, plot the 2D embedding colored by species. Describe how the cluster structure changes with perplexity.

5 Kernel PCA Experiment:

Generate data using `make_moons(n_samples=500, noise=0.1)`. Apply standard PCA and Kernel PCA (RBF, $\gamma = 15$) with 2 components. Compare the results. Why does PCA fail here?

6 Feature Engineering Pipeline:

Given a dataset with columns: `timestamp`, `user_id`, `purchase_amount`, `product_category`, `user_age`, write Python code to construct the following features:

- Cyclical encoding of the hour and day-of-week
- Per-user aggregates: total spend, average spend, number of transactions
- Target encoding for `product_category` with smoothing
- Log-transform of `purchase_amount`
- Interaction: `user_age` $\times$ `log_purchase_amount`

7 End-to-End Comparison:

On the Wine dataset (`sklearn.datasets.load_wine`), compare PCA, LDA, and Factor Analysis (2 components each). Plot the 2D representations and report which gives the best visual class separation.

Q1 Solution (Factor Analysis):

$$h_1^2 = 0.90^2 + 0.20^2 = 0.81 + 0.04 = \mathbf{0.85}$$

$$h_2^2 = 0.75^2 + 0.35^2 = 0.5625 + 0.1225 = \mathbf{0.685}$$

$$h_3^2 = 0.10^2 + 0.85^2 = 0.01 + 0.7225 = \mathbf{0.7325}$$

$$\text{Uniqueness: } \psi_1 = 0.15, \psi_2 = 0.315, \psi_3 = 0.2675$$

$$\hat{r}_{12} = (0.90)(0.75) + (0.20)(0.35) = 0.675 + 0.070 = \mathbf{0.745}$$

Q2 Solution (LDA):

$$\mathbf{S}_W^{-1} = \frac{1}{4 \cdot 2 - 1 \cdot 1} \begin{pmatrix} 2 & -1 \\ -1 & 4 \end{pmatrix} = \frac{1}{7} \begin{pmatrix} 2 & -1 \\ -1 & 4 \end{pmatrix}$$

$$\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2 = \begin{pmatrix} -4 \\ -2 \end{pmatrix}$$

$$\mathbf{w} = \frac{1}{7} \begin{pmatrix} 2 & -1 \\ -1 & 4 \end{pmatrix} \begin{pmatrix} -4 \\ -2 \end{pmatrix} = \frac{1}{7} \begin{pmatrix} -6 \\ -4 \end{pmatrix} = \begin{pmatrix} -0.857 \\ -0.571 \end{pmatrix}$$

$$z_{\text{new}} = (-0.857)(4) + (-0.571)(4) \\ = -3.428 - 2.284 = \mathbf{-5.712}$$

Interpretation of Q2

Compare $z_{\text{new}} = -5.712$ with the projected class means: $z_{\mu_1} = (-0.857)(2) + (-0.571)(3) = -3.427$ and $z_{\mu_2} = (-0.857)(6) + (-0.571)(5) = -7.997$. Since -5.712 is closer to z_{μ_2} , the new point would be classified as **Class 2**.

Q3 Solution (RBF Kernel Matrix):

$$k(\mathbf{x}, \mathbf{y}) = \exp(-0.5\|\mathbf{x} - \mathbf{y}\|^2)$$

Pairwise distances:

$$\|\mathbf{x}_1 - \mathbf{x}_2\|^2 = (1 - 3)^2 + (2 - 4)^2 = 4 + 4 = 8$$

$$\|\mathbf{x}_1 - \mathbf{x}_3\|^2 = (1 - 0)^2 + (2 - 1)^2 = 1 + 1 = 2$$

$$\|\mathbf{x}_2 - \mathbf{x}_3\|^2 = (3 - 0)^2 + (4 - 1)^2 = 9 + 9 = 18$$

Kernel values:

$$k(\mathbf{x}_1, \mathbf{x}_1) = e^0 = 1$$

$$k(\mathbf{x}_1, \mathbf{x}_2) = e^{-0.5 \times 8} = e^{-4} \approx 0.0183$$

$$k(\mathbf{x}_1, \mathbf{x}_3) = e^{-0.5 \times 2} = e^{-1} \approx 0.3679$$

$$k(\mathbf{x}_2, \mathbf{x}_3) = e^{-0.5 \times 18} = e^{-9} \approx 0.0001$$

Full Kernel Matrix:

$$\mathbf{K} = \begin{pmatrix} 1.0000 & 0.0183 & 0.3679 \\ 0.0183 & 1.0000 & 0.0001 \\ 0.3679 & 0.0001 & 1.0000 \end{pmatrix}$$

Interpretation:

- $K_{13} = 0.37$: $\mathbf{x}_1$ and $\mathbf{x}_3$ are fairly similar.
- $K_{12} = 0.02$: $\mathbf{x}_1$ and $\mathbf{x}_2$ are dissimilar.
- $K_{23} \approx 0$: $\mathbf{x}_2$ and $\mathbf{x}_3$ are very far apart.

Note

$\mathbf{K}$ is always **symmetric** ($K_{ij} = K_{ji}$) and **positive semi-definite**. Diagonal entries are always 1 for the RBF kernel. Eigendecomposing $\mathbf{K}$ gives the Kernel PCA components.

Extraction Methods

- **FA:** latent generative model, $\Sigma = \Lambda\Lambda^T + \Psi$, rotation for interpretability.
- **LDA:** supervised, max $J = S_B/S_W$, at most $g - 1$ dimensions.
- **Kernel PCA:** non-linear via kernel trick, eigendecompose $\mathbf{K}$ ($n \times n$).
- **t-SNE:** visualization only, t-distribution for crowding, perplexity critical.

Feature Engineering

- Interactions: products, ratios, differences between features.
- Row-wise aggregates: mean, std, skew, kurtosis across feature groups.
- Power transforms: Box-Cox / Yeo-Johnson for normality.
- Target encoding with smoothing to prevent leakage.
- Mahalanobis distance as a multivariate outlier feature.

Final Takeaway

These methods form a toolkit: FA for understanding, LDA for classification, Kernel PCA for non-linear structure, t-SNE for visualization, and feature engineering for model performance. Master when to use each.

Thank You!

